



Universidad Zaragoza

UNIVERSIDAD DE ZARAGOZA
ESCUELA DE INGENIERÍA Y ARQUITECTURA

Informe Final - Laboratorio de ingeniería del software

*Sistema de reserva y gestión de espacios del edificio Ada
Byron.*

Autores:

Mario Caudevila Ruiz 870421
Víctor Martínez Páramo 875325
Pablo Calvo Izquierdo 831754

Grado:

Ingeniería Informática

Profesor:

Dr. Rubén Béjar Hernández

Zaragoza, España
29 de mayo de 2026

Índice

1. Introducción	3
1.1. Descripción del proyecto	3
1.2. El equipo	3
1.3. Estructura del documento	3
2. Requisitos	5
2.1. Diccionario de datos	5
2.2. Catálogo de requisitos	6
2.2.1. Requisitos funcionales de los usuarios	6
2.2.2. Requisitos funcionales de los usuarios gerentes	6
2.2.3. Requisitos funcionales API	7
2.2.4. Requisitos funcionales del sistema	7
2.2.5. Requisitos funcionales opcionales	9
2.2.6. Requisitos no funcionales	9
2.3. Gestión de roles y espacios	10
2.3.1. Tabla de roles	10
2.3.2. Tabla de espacios	10
2.3.3. Tabla de roles y espacios	11
2.4. Ciclo de vida de una reserva	11
2.5. Bocetos GUI	13
3. Diseño e Implementación	17
3.1. Estado actual de la aplicación	17
3.1.1. Funcionalidades implementadas	17
3.1.2. GUI en su estado actual	18
3.1.3. Problemas hallados	22
3.1.4. Aspectos de usabilidad	23
3.2. Arquitectura del sistema	24
3.2.1. Vista de módulos	24
3.2.2. Vista de componentes y conectores	26
3.2.3. Vista de despliegue	28
3.3. Diseño Dirigido por el Dominio	30
3.3.1. Diagrama de clases de la capa de dominio	30
3.3.2. Entidades, objetos valor y agregados	31
3.3.3. Factorías y repositorios	32
3.3.4. Servicios, interfaces reveladoras, aserciones y funciones sin efectos secundarios	33
3.3.5. Decisiones de diseño relevantes	34
3.4. Implementación del SIG	34
3.5. Documentación de la API	35
3.6. Tests y cobertura	38
3.6.1. Estrategia de testing	38
3.6.2. Tests unitarios del app-server	39
3.6.3. Tests de integración del gateway	39
3.6.4. Cobertura	39

3.7.	Calidad y construcción automática	41
3.7.1.	GitHub Actions CI	41
3.7.2.	Estrategia de ramas y definición de hecho	41
3.7.3.	Gestión de issues	42
4.	Conclusiones	43
4.1.	Resumen	43
4.2.	Cumplimiento de objetivos	43
4.3.	Horas dedicadas por persona	44

1. Introducción

1.1. Descripción del proyecto

ByronSpace es una aplicación web para la gestión y reserva de espacios del edificio Ada Byron, perteneciente a la Escuela de Ingeniería y Arquitectura (EINA) de la Universidad de Zaragoza. El edificio alberga distintos tipos de espacios: aulas, laboratorios, seminarios, despachos y salas comunes, cuya gestión manual resulta ineficiente y propensa a conflictos de disponibilidad.

El proyecto nace de la necesidad de digitalizar y centralizar la gestión de estos espacios, garantizando el cumplimiento de las reglas de uso según el rol del solicitante y ofreciendo a la gerencia herramientas de supervisión y control sobre todas las reservas activas del sistema. La aplicación permite a los distintos usuarios del edificio (estudiantes, investigadores, docentes, técnicos de laboratorio, conserjes y gerentes) consultar la disponibilidad de los espacios, buscarlos por distintos criterios y realizar reservas a través de una interfaz web con mapa interactivo. Cada rol tiene permisos diferenciados sobre qué tipos de espacios puede reservar y bajo qué condiciones.

La aplicación está orientada a usuarios finales a través de una interfaz gráfica moderna e intuitiva. El backend expone una API REST que sirve tanto a la interfaz web como a cualquier cliente externo que quiera integrarse con el sistema.

Además de la funcionalidad requerida, se han implementado cuatro puntos de funcionalidad opcional (O1, O2, O3 y O7) que amplían las capacidades del sistema en aspectos como la configuración de ocupación por espacio, la visualización sobre mapa y la gestión de nuevos roles y tipos de asignación de despachos. Estos puntos se describen en detalle en el capítulo de requisitos.

1.2. El equipo

El proyecto ha sido desarrollado por el Equipo 06 de la asignatura Laboratorio de Ingeniería del Software, perteneciente al Grado en Ingeniería Informática de la Universidad de Zaragoza, formado por los siguientes miembros:

- Mario Caudevilla Ruiz (870421)
- Víctor Martínez Páramo (875325)
- Pablo Calvo Izquierdo (831754)

La organización del proyecto en GitHub, donde se alojan todos los repositorios del sistema, se encuentra en la siguiente URL:
<https://github.com/UNIZAR-30249-2026-RESERVASADA>

1.3. Estructura del documento

El presente documento se organiza en cuatro capítulos, precedidos de una portada identificativa y un índice de contenidos.

El capítulo 1 presenta el proyecto, el equipo de desarrollo y la estructura del propio documento.

El capítulo 2 recoge los requisitos del sistema, incluyendo el diccionario de datos, el catálogo de requisitos funcionales y no funcionales, las tablas de roles y espacios, el ciclo de vida de una reserva y los bocetos de la interfaz gráfica.

El capítulo 3 describe el diseño y la implementación del sistema, con especial atención a la arquitectura, el modelo de dominio siguiendo los principios del Diseño Dirigido por el Dominio, la implementación del sistema de información geográfica, la documentación de la API, la estrategia de tests y las medidas de calidad adoptadas.

El capítulo 4 recoge las conclusiones del proyecto, el grado de cumplimiento de los objetivos de evaluación y las horas dedicadas por cada miembro del equipo.

2. Requisitos

Los requisitos del sistema se han obtenido a partir del enunciado del proyecto y se han clasificado en requisitos funcionales de usuario, de gerente, de API, del sistema, opcionales y no funcionales. Para resolver ambigüedades e incompletitudes del enunciado, el equipo ha tomado decisiones explícitas que se documentan en las secciones correspondientes.

2.1. Diccionario de datos

A continuación se definen los términos del dominio utilizados a lo largo del documento y referenciados en el catálogo de requisitos.

- **(1) Espacios Disponibles:** Salas disponibles, utilizables o reservables del edificio Ada Byron.
- **(2) Información Personal:** Datos necesarios de cada persona para poder identificarse.
- **(3) Roles:** Estudiante, investigador contratado, docente-investigador, conserje, técnico de laboratorio, gerente e investigador visitante.
- **(4) Departamentos:** Informática e Ingeniería de Sistemas e Ingeniería Electrónica y Comunicaciones.
- **(5) Categoría de reserva:** Aula, seminario, laboratorio, despacho y sala común.
- **(6) Horario de reserva disponible:** Horario en el que una sala reservable del edificio Ada Byron está disponible para su uso.
- **(7) Titular asignado:** La EINA, un departamento (4), o usuarios investigadores contratados o docentes-investigadores.
- **(8) Criterios:** Identificador, categoría, ocupantes máximos y en qué planta se encuentra.
- **(9) Información de reserva:** Tipo de uso, número máximo de personas en total, hora de inicio, duración y texto de explicaciones y detalles adicionales.
- **(10) Reservas periódicas:** Reserva que permite asegurar un mismo espacio disponible (1) en el mismo horario y día de la semana, mediante un único procedimiento de reserva.
- **(11) Reservas solapadas:** Reservas que coinciden en el mismo periodo de tiempo de manera total o parcial.
- **(12) Reservas vivas:** Reservas cuya hora de finalización es posterior a la hora de consulta.
- **(13) WMS:** Web Map Service.
- **(14) Calendario:** Días y horas en las que se encuentra abierto el edificio.

2.2. Catálogo de requisitos

2.2.1. Requisitos funcionales de los usuarios

Código	Descripción
RFU1 (A)	El sistema debe permitir al usuario interactuar con el mapa de los espacios disponibles (1).
RFU2 (B)	El sistema debe permitir al usuario iniciar sesión mediante la información personal (2) introducida.
RFU3 (D)	El sistema debe permitir a los usuarios realizar reservas de los espacios disponibles (1) reservables.
RFU4 (D)	El sistema debe permitir a los usuarios buscar espacios disponibles (1) reservables según diferentes criterios (8).
RFU5 (D)	El sistema debe permitir a los usuarios seleccionar uno o más espacios disponibles (1) tras realizar una búsqueda.
RFU6	El sistema debe permitir a los usuarios cancelar sus propias reservas activas.

Tabla 1: Tabla de requisitos funcionales de usuarios

2.2.2. Requisitos funcionales de los usuarios gerentes

Código	Descripción
RFG1 (B)	El sistema debe permitir a los usuarios gerentes asignar roles (3) a otros usuarios.
RFG2 (B)	El sistema debe permitir a los usuarios gerentes disponer del rol docente investigador.
RFG3 (B)	El sistema debe permitir a los usuarios gerentes modificar el rol (3) de otros usuarios.
RFG4 (B)	El sistema debe restringir a los usuarios gerentes sin rol de docente-investigador estar adscritos a un departamento (4).
RFG5 (C)	El sistema debe permitir a los usuarios gerentes establecer si el espacio disponible (1) es reservable.
RFG6 (C)	El sistema debe permitir a los usuarios gerentes modificar la categoría de reserva (5) de los espacios disponibles (1) reservables.
RFG7 (C)	El sistema debe permitir a los usuarios gerentes modificar el horario de reserva disponible (6) de los espacios (1) disponibles reservables.
RFG8 (C)	El sistema debe permitir a los usuarios gerentes modificar el titular asignado (7) al espacio disponible (1), según la categoría de reserva (5) de cada espacio.
RFG9 (H)	El sistema debe permitir a los usuarios gerentes consultar todas las reservas vivas (12).
RFG10 (H)	El sistema debe permitir a los usuarios gerentes eliminar cualquier reserva activa, siempre que no esté en curso y falten más de 24 horas para su inicio.

Tabla 2: Tabla de requisitos funcionales de usuarios gerentes

2.2.3. Requisitos funcionales API

Código	Descripción
RFA1 (B)	El sistema debe permitir a los usuarios gerentes modificar el departamento (4) al que están adscritos otros usuarios.
RFA2	El sistema debe permitir a los usuarios gerentes modificar el rol (3) de otros usuarios.
RFA3	El sistema debe permitir a los usuarios gerentes añadir personas nuevas.
RFA4	El sistema debe permitir a los usuarios gerentes modificar el calendario (14) y porcentaje de ocupación del edificio Ada Byron.

Tabla 3: Tabla de requisitos funcionales de API

2.2.4. Requisitos funcionales del sistema

Código	Descripción
RF1 (A)	El sistema debe mostrar un mapa del edificio Ada Byron superpuesto a un mapa de base urbana.
RF2 (A)	El sistema debe permitir distinguir el tipo de espacios disponibles (1) mediante colores.
RF3 (B)	El sistema debe permitir a los usuarios tener un único rol (3) asignado excepto a los gerentes que también pueden tener asignado el rol de docente-investigador.
RF4 (B)	El sistema debe restringir la creación de nuevos departamentos (4).
RF5 (B)	El sistema debe restringir la modificación de departamentos (4).
RF6 (B)	El sistema debe permitir que los usuarios con rol (3) de investigador contratado, docente-investigador o técnico de laboratorio estén adscritos a un único departamento (4).
RF7 (B)	El sistema debe restringir que los usuarios estudiantes y conserjes estén adscritos a un departamento (4).
RF8 (C)	El sistema debe establecer el horario de apertura del edificio Ada Byron como el horario de reserva disponible (6) por defecto de los espacios disponibles (1) reservables.
RF9 (C)	El sistema debe asignar las aulas y salas comunes a la EINA.
RF10 (C)	El sistema debe asignar los despachos a un departamento (4) o a un usuario investigador contratado o docente-investigador.
RF11 (C)	El sistema debe asignar los seminarios y laboratorios a un departamento (4) o a la EINA.
RF12 (C)	El sistema debe actualizar el porcentaje de uso máximo de los espacios disponibles (1) cuando se modifique el porcentaje de ocupación del edificio Ada Byron. El gerente puede elegir entre dos modos: aplicar el cambio solo a los espacios que heredan el porcentaje del edificio, o aplicarlo a todos los espacios sobrescribiendo también los valores individuales. Los espacios con porcentaje propio se gestionan individualmente mediante RFO1.

Continúa en la siguiente página

Código	Descripción
RF13 (C)	El sistema debe restringir la modificación del número máximo de ocupantes del espacio disponible (1) a los usuarios gerentes.
RF14 (E)	El sistema debe restringir a los usuarios de reservar uno o más espacios disponibles (1) reservables en caso de no proporcionar toda la información de reserva (9).
RF15 (E)	El sistema debe restringir a los usuarios las reservas periódicas (10) de espacios disponibles (1) reservables.
RF16 (E)	El sistema debe restringir a los usuarios la creación de reservas cuya fecha de finalización sea anterior o igual a la fecha de inicio.
RF17 (F)	El sistema debe restringir a los usuarios estudiantes la reserva de espacios que no sean salas comunes.
RF18 (F)	El sistema debe restringir a los usuarios estudiantes y técnicos de laboratorio la reserva de aulas.
RF19 (F)	El sistema debe restringir a los usuarios estudiantes la reserva de laboratorios.
RF20 (F)	El sistema debe restringir a los usuarios la reserva de un laboratorio adscrito a un departamento (4) distinto al suyo.
RF21 (F)	El sistema debe restringir la reserva de despachos.
RF22 (F)	El sistema debe restringir las reservas cuya ocupación es superior a la ocupación máxima permitida por el espacio solicitado, teniendo en cuenta el porcentaje máximo de ocupación.
RF23 (F)	El sistema debe restringir las reservas solapadas (11).
RF24 (F)	El sistema debe permitir a los usuarios gerentes reservar cualquier espacio disponible (1) reservable, siempre que no sea una reserva solapada (11).
RF25 (G)	El sistema debe indicar a los usuarios si una reserva es posible o no en el momento de su realización, y mostrar el motivo en caso negativo.
RF26 (H)	El sistema debe notificar a los usuarios la eliminación de cualquiera de sus reservas.
RF27 (I)	El sistema debe cancelar automáticamente las reservas existentes que dejen de ser válidas cuando se produzca cualquier modificación estructural del sistema, siempre que falten más de 24 horas para su inicio.
RF27.1 (I)	Se consideran cambios estructurales: el cambio de categoría de un espacio, el cambio de reservable a false, la modificación del porcentaje de ocupación, el cambio de horario del espacio o del edificio, y el cambio de rol o departamento del usuario.
RF28 (E)	El sistema debe restringir la creación de reservas con menos de 24 horas de antelación respecto a la hora de inicio.

Tabla 4: Tabla de requisitos funcionales del sistema

2.2.5. Requisitos funcionales opcionales

Código	Descripción
RFO1 (OP1)	El sistema debe permitir a los usuarios gerentes modificar de manera individual el porcentaje de ocupación máxima de los espacios disponibles (1) reservables.
RFO2 (OP2)	El sistema debe mostrar gráficamente los resultados de búsqueda realizadas por los usuarios en el mapa que se muestra.
RFO3 (OP3)	El sistema debe permitir a usuarios gerentes, investigadores contratados y docentes-investigadores la reserva de despachos que estén asignados a su mismo departamento (4).
RFO4.1 (OP7)	El sistema debe permitir a los usuarios investigadores visitantes estar adscritos a un departamento (4).
RFO4.2 (OP7)	El sistema debe permitir asignar un despacho a los usuarios investigadores visitantes.
RFO4.3 (OP7)	El sistema debe restringir la modificación de rol (3) de los usuarios investigadores visitantes.
RFO4.4 (OP7)	El sistema debe permitir la reserva de un despacho asignado a un usuario investigador visitante a usuarios investigadores contratados y docentes-investigadores de su mismo departamento (4).

Tabla 5: Tabla de requisitos funcionales opcionales

2.2.6. Requisitos no funcionales

Código	Descripción
RNF1	El mapa que contiene la aplicación es de tipo WMS (13).
RNF2	El sistema debe asegurar que las modificaciones simultáneas realizadas por dos o más usuarios se guardan correctamente.
RNF3	El sistema debe informar al usuario qué rol tiene en cada momento.
RNF4	La arquitectura del sistema es hexagonal.

Tabla 6: Tabla de requisitos no funcionales

2.3. Gestión de roles y espacios

Con el objetivo de concretar ciertas restricciones del sistema y evitar posibles ambigüedades, se presentan a continuación tres tablas que formalizan las combinaciones permitidas y prohibidas relacionadas con los roles y espacios de la aplicación. Estas tablas muestran las posibles transiciones entre roles a lo largo de la vida del sistema, las combinaciones válidas entre tipo físico de espacio y categoría de reserva, y las reglas que determinan qué roles pueden reservar cada tipo de espacio. Su objetivo es expresar de forma estructurada las reglas que condicionan el comportamiento del sistema.

2.3.1. Tabla de roles

La siguiente tabla representa un requisito funcional del sistema, expresado en forma de tabla para facilitar su comprensión. Concretamente, representa qué cambios de rol son posibles y cuáles no, permitiendo a los usuarios gerentes realizar exclusivamente las transiciones indicadas.

Es importante destacar que las transiciones de rol son unidireccionales y muy restrictivas. Estas transiciones no reflejan necesariamente el funcionamiento real del edificio Ada Byron, sino que representan las decisiones tomadas por el equipo para modelar de forma coherente una progresión académica y laboral razonable dentro del sistema. Un estudiante puede convertirse en investigador contratado al incorporarse a un proyecto de investigación, y este a su vez puede ascender a docente-investigador. En el ámbito técnico, un técnico de laboratorio puede pasar a conserje y viceversa. El rol de investigador visitante es permanente durante la vida del sistema y no puede cambiar. El flag de gerente es independiente del rol y puede asignarse a un usuario docente investigador o a un usuario sin rol, permitiendo así que un gerente ejerza también funciones docentes o actúe como gerente puro.

De/A	Estudiante	Investigador	Docente-Invest.	Técnico Lab	Conserje	Gerente	Inv. Visitante
Estudiante	—	Sí	No	No	No	No	No
Investigador	No	—	Sí	No	No	No	No
Docente-Invest.	No	No	—	No	No	No	No
Técnico Lab	No	No	No	—	Sí	No	No
Conserje	No	No	No	Sí	—	No	No
Gerente	No	No	No	No	No	—	No
Inv. Visitante	No	No	No	No	No	No	—

Tabla 7: Tabla de transiciones de roles

2.3.2. Tabla de espacios

De igual manera que la tabla anterior, esta tabla también representa un requisito. En este caso representa los cambios de categoría de reserva que pueden realizar los usuarios gerentes.

Es importante destacar que las transiciones permitidas se calculan a partir del tipo físico del espacio, es decir, su uso original registrado en la base de datos geográfica, y no de la categoría de reserva actual. Esto significa que un espacio cuyo tipo físico es laboratorio pero que tiene asignada temporalmente la categoría ".aula" puede volver a la categoría "laboratorio." aunque esa transición no aparezca en la tabla, ya que su tipo

físico lo permite. La única categoría que nunca puede cambiar es "despacho", independientemente del tipo físico del espacio.

De/A	Aula	Laboratorio	Seminario	Despacho	Sala común
Aula	—	No	Sí	No	Sí
Laboratorio	Sí	—	Sí	No	No
Seminario	Sí	No	—	No	Sí
Despacho	No	No	No	—	No
Sala común	Sí	No	Sí	No	—

Tabla 8: Tabla de transiciones de categorías de espacio

2.3.3. Tabla de roles y espacios

Por último, esta tabla también muestra un requisito. En ella, se muestra qué espacios con una categoría de reserva en concreto, pueden ser reservados por cada usuario con un rol en específico.

En la tabla, "Despacho (dpto)" hace referencia a un despacho asignado a un departamento (requisito O3), mientras que "Despacho (inv. visitante)" hace referencia a un despacho asignado a un investigador visitante (requisito O7). Cuando una celda indica "Solo su dpto", significa que el usuario solo puede reservar ese espacio si pertenece al mismo departamento al que está asignado el espacio, ya sea un laboratorio adscrito a ese departamento, un despacho asignado a ese departamento o un despacho cuyo investigador visitante pertenece a ese departamento.

Rol / Espacio	Aula	Laboratorio	Seminario	Sala común	Despacho (dpto)	Despacho (inv. visitante)
Estudiante	No	No	No	Sí	No	No
Investigador	Sí	Solo su dpto	Sí	Sí	Solo su dpto	Solo su dpto
Docente-Invest.	Sí	Solo su dpto	Sí	Sí	Solo su dpto	Solo su dpto
Técnico Lab	No	Solo su dpto	Sí	Sí	No	No
Conserje	Sí	Sí	Sí	Sí	No	No
Gerente	Sí	Sí	Sí	Sí	Sí	Sí
Inv. Visitante	Sí	Solo su dpto	Sí	Sí	No	No

Tabla 9: Tabla de roles y espacios reservables

2.4. Ciclo de vida de una reserva

Para expresar el comportamiento del sistema en relación con las reservas, el equipo ha modelado el ciclo de vida de estas mediante un autómata finito. Este modelo permite definir claramente los estados persistentes de una reserva y las transiciones que ocurren en función de las validaciones y eventos relevantes del sistema.

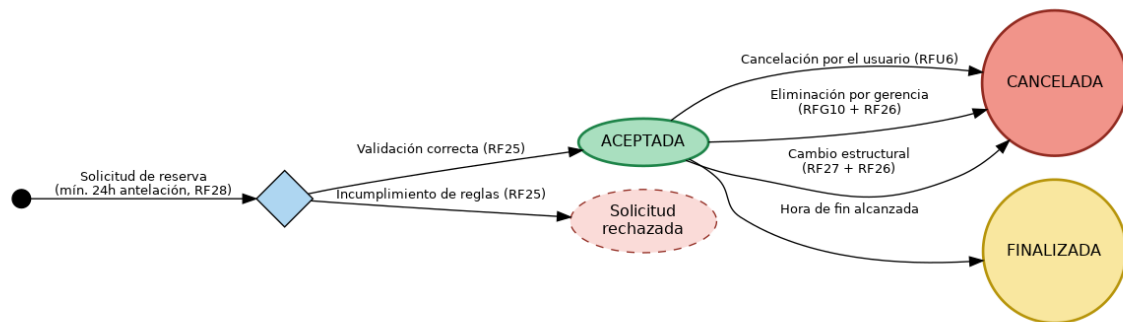


Figura 1: Autómata finito del ciclo de vida de una reserva

El proceso se inicia cuando un usuario envía una solicitud de reserva. En ese momento el sistema evalúa de forma instantánea si la solicitud cumple todas las restricciones funcionales establecidas, incluyendo que la reserva se realice con al menos 24 horas de antelación respecto a su hora de inicio. Esta evaluación se representa mediante un pseudoestado de decisión y no constituye un estado persistente de la reserva.

Si las condiciones se cumplen, la reserva pasa al estado **ACEPTADA**, considerándose válida y activa hasta su hora de finalización o hasta que se produzca su cancelación. En caso contrario, la solicitud es rechazada y se informa al usuario del motivo, sin que la reserva llegue a persistirse en el sistema.

El estado representado con líneas discontinuas "Solicitud rechazada" no constituye un estado persistente del sistema. Se incluye en el diagrama únicamente para representar visualmente el resultado de una validación fallida. En ese caso el sistema devuelve un error descriptivo al usuario indicando el motivo del rechazo, pero la reserva no llega a persistirse en ningún momento.

Desde el estado **ACEPTADA**, la reserva puede transitar a dos estados finales. El estado **CANCELADA** se alcanza por tres vías: cancelación voluntaria por parte del propio usuario que realizó la reserva, sin restricción de antelación y sin notificación; eliminación manual por parte de la gerencia, siempre que la reserva no esté en curso y falten más de 24 horas para su inicio, en cuyo caso el usuario sí recibe una notificación; o de forma automática cuando un cambio estructural del sistema provoca que la reserva deje de cumplir las reglas establecidas, notificando también al usuario en ese caso. Se consideran cambios estructurales los siguientes:

- El cambio de categoría de un espacio que impida al usuario reservarlo con su rol actual.
- El cambio de reservable a false en un espacio.
- La modificación del porcentaje de ocupación que provoque que el número de personas de la reserva supere el nuevo aforo permitido.
- El cambio de horario del espacio o del edificio que deje la reserva fuera del nuevo rango horario.

- El cambio de rol o departamento del usuario que le impida reservar ese espacio.

Cabe destacar que si el cambio estructural afecta a una reserva con menos de 24 horas de antelación, dicha reserva no se cancela y se respeta su validez original. El estado FINALIZADA se alcanza cuando la hora de fin de la reserva es superada por el tiempo actual.

El modelo garantiza que cada reserva se encuentre en todo momento en un único estado persistente y que cualquier evento relevante produzca una transición controlada y coherente.

2.5. Bocetos GUI

Antes de comenzar el desarrollo, el equipo diseñó una serie de bocetos de la interfaz de usuario con Figma para tener una idea clara de cómo iba a quedar la aplicación y qué pantallas iba a necesitar.

Los bocetos recogen las pantallas principales de la aplicación: el mapa interactivo de espacios, el formulario de reserva, el panel con las reservas propias de cada usuario, el panel de reservas vivas accesible para el gerente con todas las reservas activas del sistema, y el panel de administración del gerente. A continuación se muestran los bocetos elaborados.

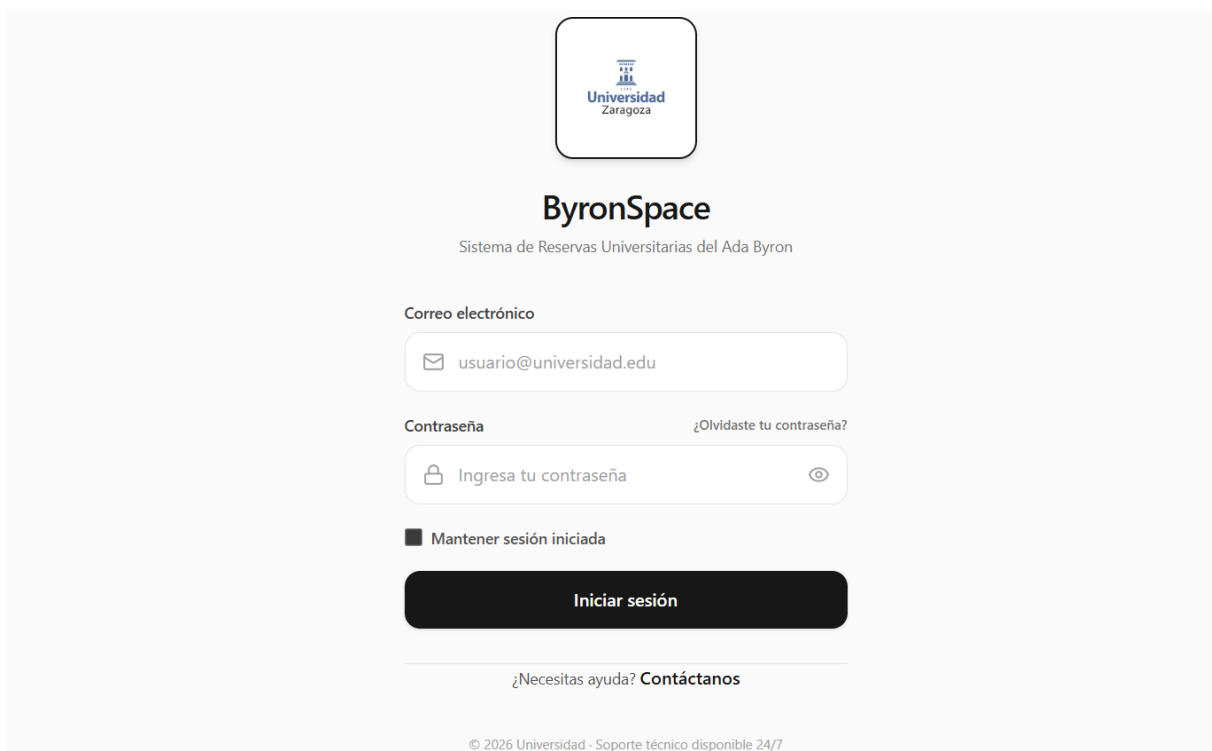


Figura 2: Boceto de la pantalla de inicio de sesión

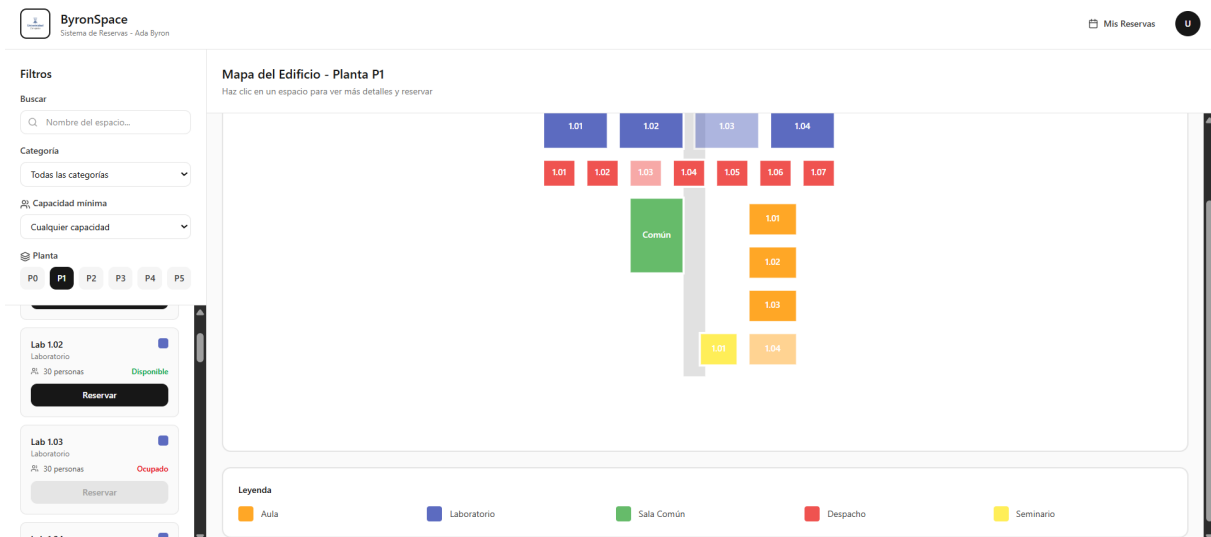


Figura 3: Boceto de la pantalla principal con mapa interactivo

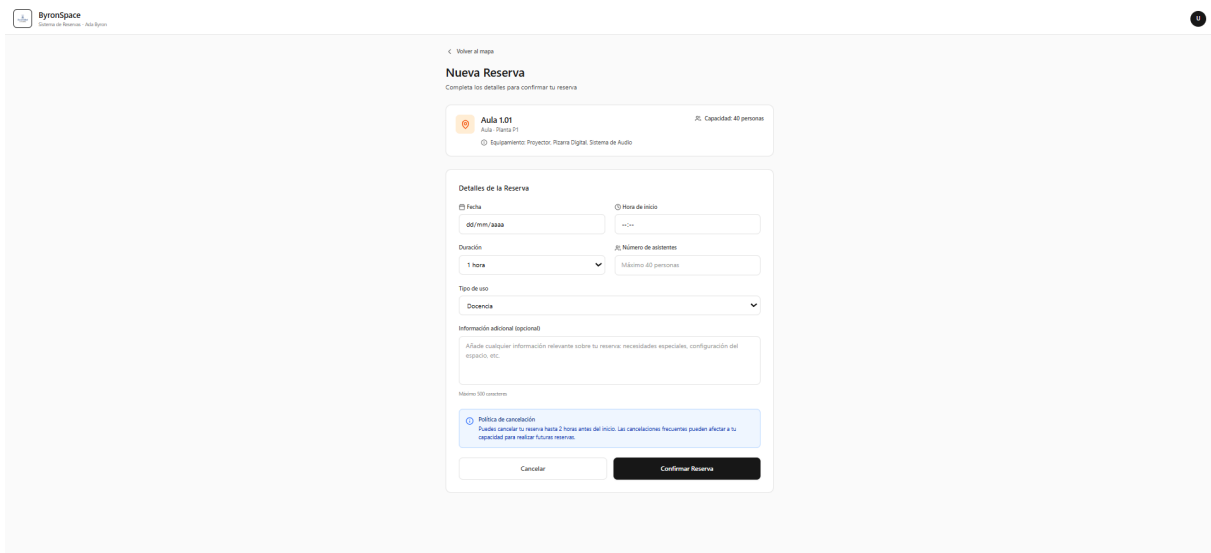


Figura 4: Boceto del formulario de reserva

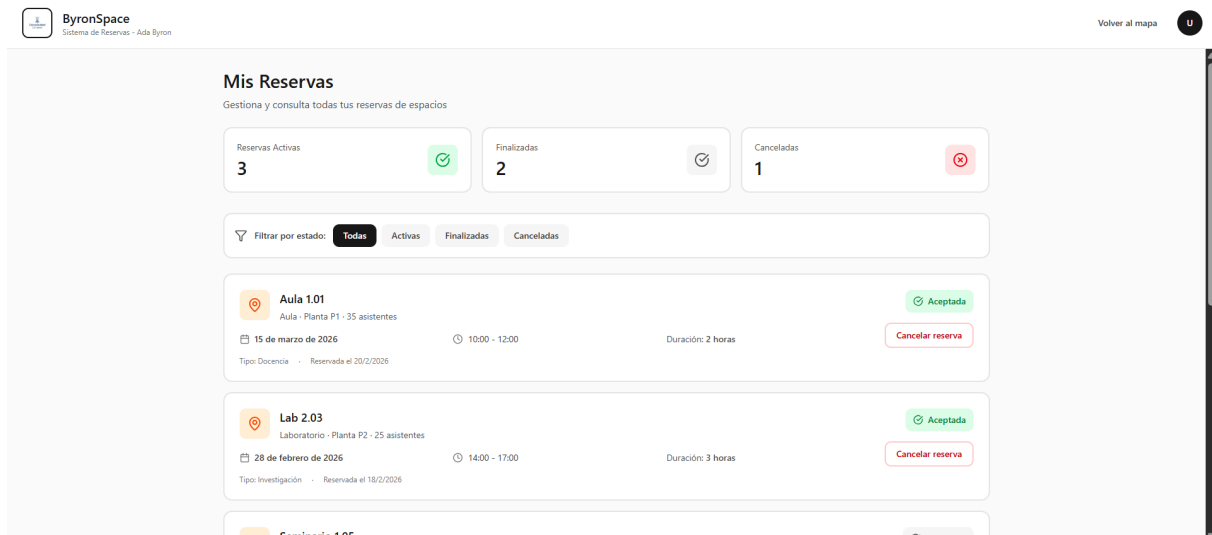


Figura 5: Boceto del panel de reservas del usuario

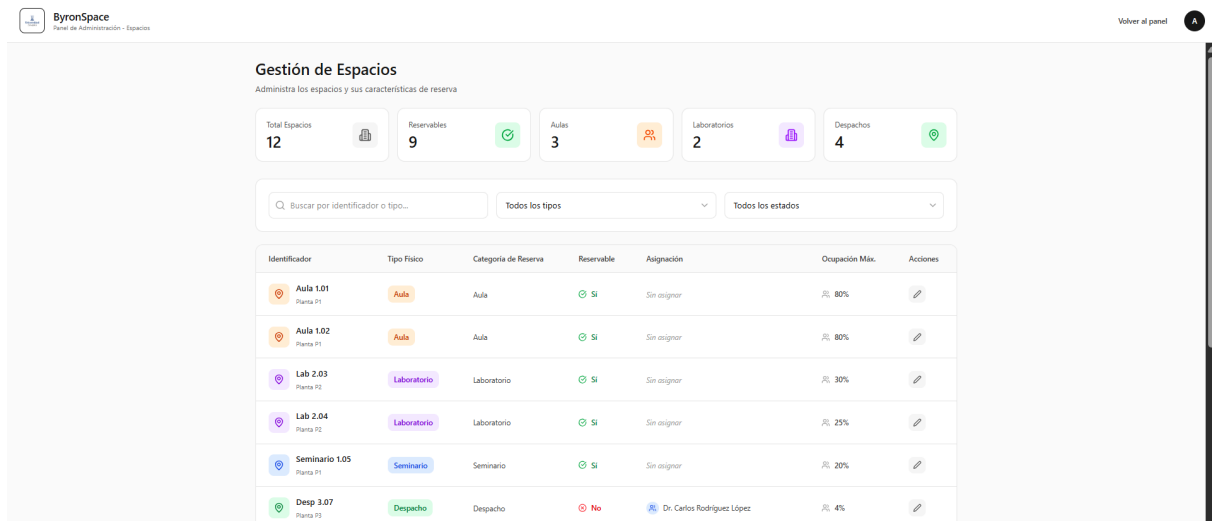


Figura 6: Boceto del panel de gestión de espacios

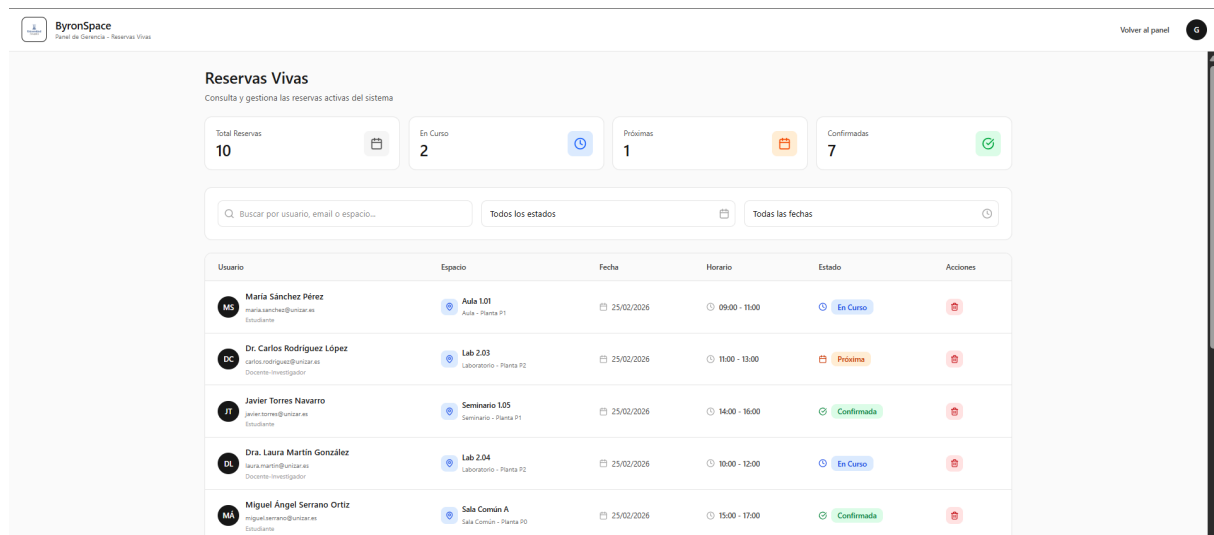


Figura 7: Boceto del panel de reservas vivas del gerente

3. Diseño e Implementación

3.1. Estado actual de la aplicación

3.1.1. Funcionalidades implementadas

La aplicación se encuentra en un estado de desarrollo completo, con toda la funcionalidad requerida implementada y operativa, así como los cuatro puntos de funcionalidad opcional seleccionados. A continuación se detalla el estado de cada área funcional.

Gestión de usuarios y roles

El sistema permite crear usuarios con distintos roles y gestionar sus transiciones según las reglas definidas. Un usuario gerente puede crear nuevos usuarios, modificar su rol y departamento, y activar o desactivar el flag de gerente. Las validaciones de compatibilidad entre rol, departamento y flag de gerente se aplican tanto en el dominio como en los casos de uso. Cualquier cambio de rol o departamento desencadena además la invalidación automática de las reservas que dejen de ser válidas con el nuevo perfil del usuario, respetando siempre la restricción de las 24 horas de antelación.

Reserva de espacios

Los usuarios pueden buscar espacios por nombre o identificador mediante una barra de búsqueda, y también filtrar por planta, categoría y número de ocupantes. El filtro de ocupantes compara el valor introducido contra el aforo efectivo de cada espacio, es decir, el aforo máximo multiplicado por el porcentaje de ocupación permitido. Por ejemplo, un espacio con aforo 40 y porcentaje del 50 % admite como máximo 20 personas, por lo que no aparecería en una búsqueda de 25 ocupantes. Los resultados se muestran de dos formas simultáneas: sobre el mapa interactivo, donde los espacios que no cumplen los criterios se ocultan o desactivan visualmente, y en una sección de resultados con tarjetas informativas de cada espacio encontrado. El usuario puede seleccionar uno o varios espacios desde cualquiera de las dos vistas para realizar la reserva. El sistema valida en el momento de la solicitud que se cumplen todas las restricciones: antelación mínima de 24 horas, día laborable, horario del espacio, permisos del rol, aforo y ausencia de solapamientos. En caso de rechazo el sistema muestra un mensaje descriptivo con el motivo concreto.

Gestión de reservas

Los usuarios pueden consultar su historial de reservas y cancelar las que estén activas. El gerente puede consultar todas las reservas vivas del sistema y eliminar cualquiera de ellas, siempre que no esté en curso y falten más de 24 horas para su inicio.

Notificaciones

El sistema genera notificaciones automáticas cuando una reserva es cancelada por el gerente o invalidada por un cambio estructural. Cuando el usuario tiene notificaciones pendientes, el icono del panel muestra un indicador numérico en rojo con el número de notificaciones no leídas. Al acceder al panel, las notificaciones se marcan automáticamente como leídas y el indicador desaparece.

Gestión de espacios y edificio

El gerente puede modificar las propiedades de cada espacio: categoría de reserva, flag

de reservable, horario propio, porcentaje de ocupación individual y asignación. También puede modificar el horario y el porcentaje de ocupación del edificio, eligiendo si el cambio afecta solo a los espacios que heredan el valor del edificio o a todos los espacios. Cualquier cambio que invalide reservas existentes las cancela automáticamente y notifica a los usuarios afectados, respetando siempre la restricción de las 24 horas de antelación.

Funcionalidad opcional

Se han implementado los puntos O1, O2, O3 y O7. El porcentaje de ocupación es configurable espacio por espacio heredando el del edificio por defecto (O1). Los resultados de búsqueda se visualizan sobre el mapa interactivo (O2). Los despachos asignados a un departamento pueden ser reservados por investigadores contratados y docentes investigadores de ese departamento (O3). Se ha añadido el rol de investigador visitante con sus reglas de asignación de despacho y reserva (O7).

3.1.2. GUI en su estado actual

A continuación se muestran capturas de pantalla de la aplicación en su estado actual, reflejando la interfaz gráfica implementada. Las pantallas recogen los principales flujos de interacción del sistema: el mapa interactivo de espacios, el buscador con sus filtros y la sección de resultados, el formulario de reserva, el panel de reservas del usuario, el panel de notificaciones y el panel de administración del gerente.

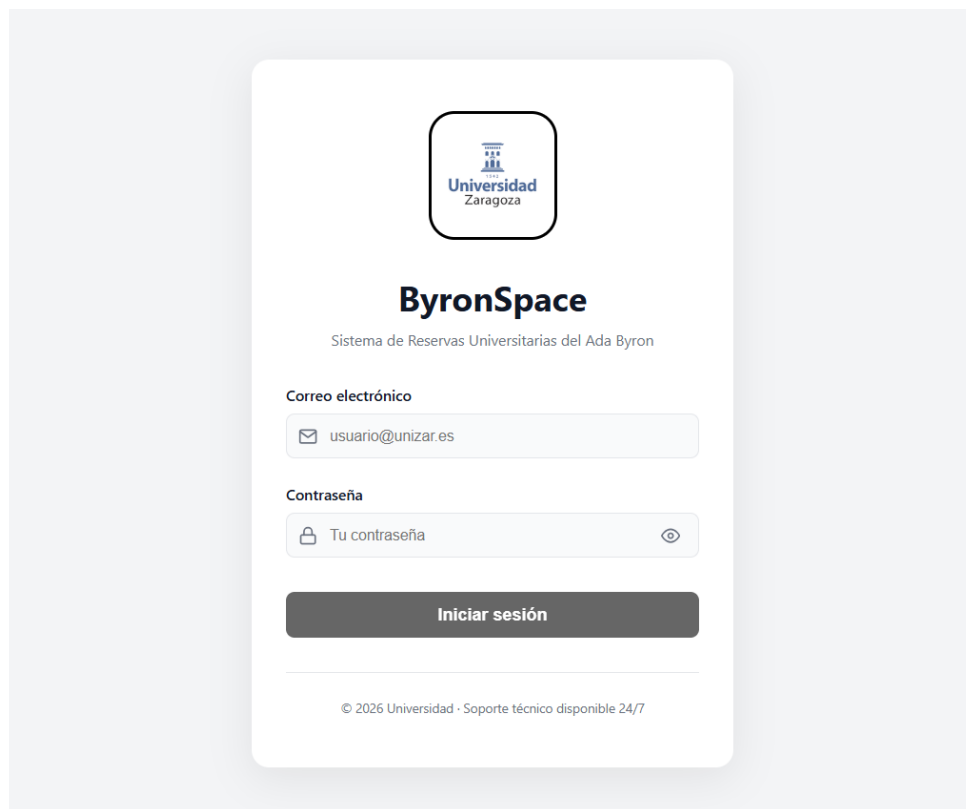


Figura 8: Pantalla de inicio de sesión

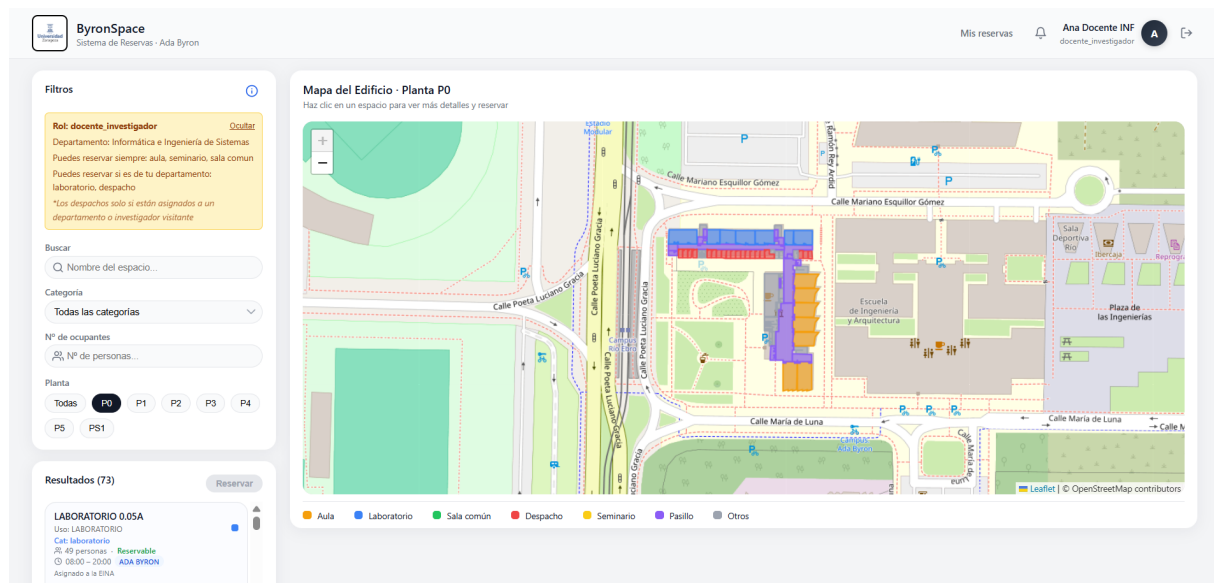


Figura 9: Pantalla principal con mapa interactivo

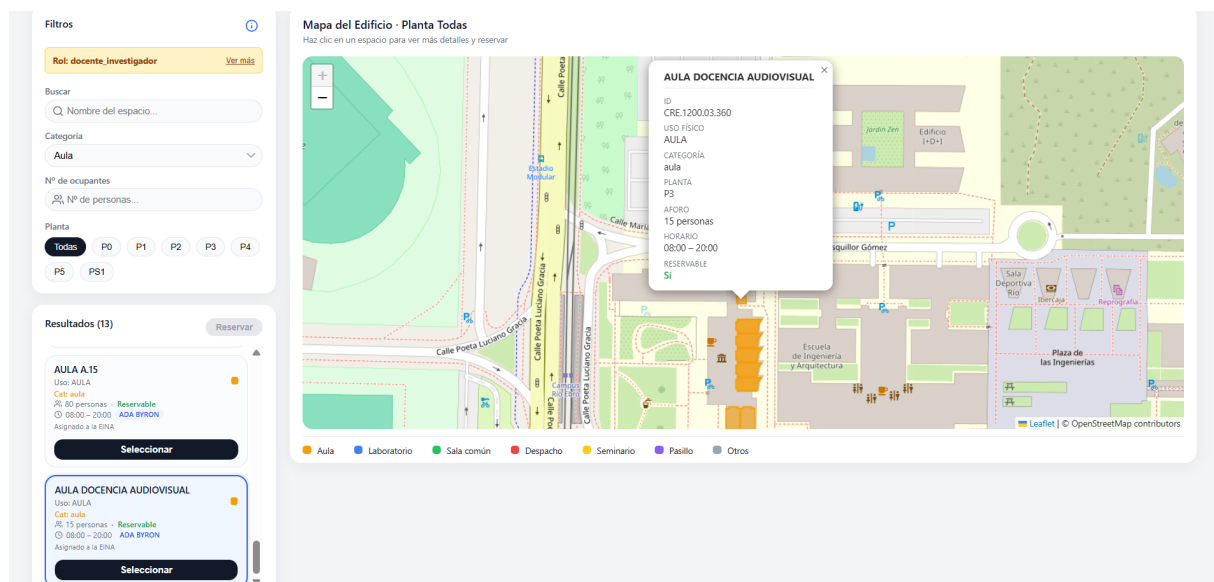


Figura 10: Pantalla principal con mapa interactivo y filtros aplicados

The screenshot shows the 'Nueva Reserva' form in the ByronSpace system. The form is titled 'Nueva Reserva' and includes a subtitle 'Completa los detalles para confirmar tu reserva'. It contains several sections: 'Espacio seleccionado' (Selected Space) showing 'AULA A02' with a capacity of 121 people; 'Detalles de la Reserva' (Reservation Details) which includes a 'Horario disponible para esta reserva' (08:00 - 20:00), a date field set to '25/05/2026', a start time field set to '14:04', a duration field set to '1', a dropdown for 'NÚMERO DE ASISTENTES POR ESPACIO' (121), a dropdown for 'TIPO DE USO' (Docencia), and an 'INFORMACIÓN ADICIONAL' (Additional Information) text area with the word 'reserva' entered. At the bottom right, there are 'Cancelar' and 'Confirmar reserva' buttons.

Figura 11: Pantalla del formulario de reserva

The screenshot shows the 'Mis Reservas' panel in the ByronSpace system. The panel is titled 'Mis Reservas' and includes a subtitle 'Gestiona y consulta todas tus reservas de espacios'. It features three summary cards: '1 Activas' (Active), '0 Finalizadas' (Finalized), and '0 Canceladas' (Cancelled). Below these cards are filter buttons: 'Todas', 'Activa', 'Cancelada', and 'Finalizada'. A list of reservations is shown, with one reservation for 'AULA A02' on '25 may 2026' from '14:04 - 15:04' for '121 personas' of type 'Docencia'. This reservation is marked as 'Activa' (Active) and has a 'Cancelar' button next to it. The total count '1 reservas' is displayed on the right.

Figura 12: Pantalla del panel de reservas del usuario

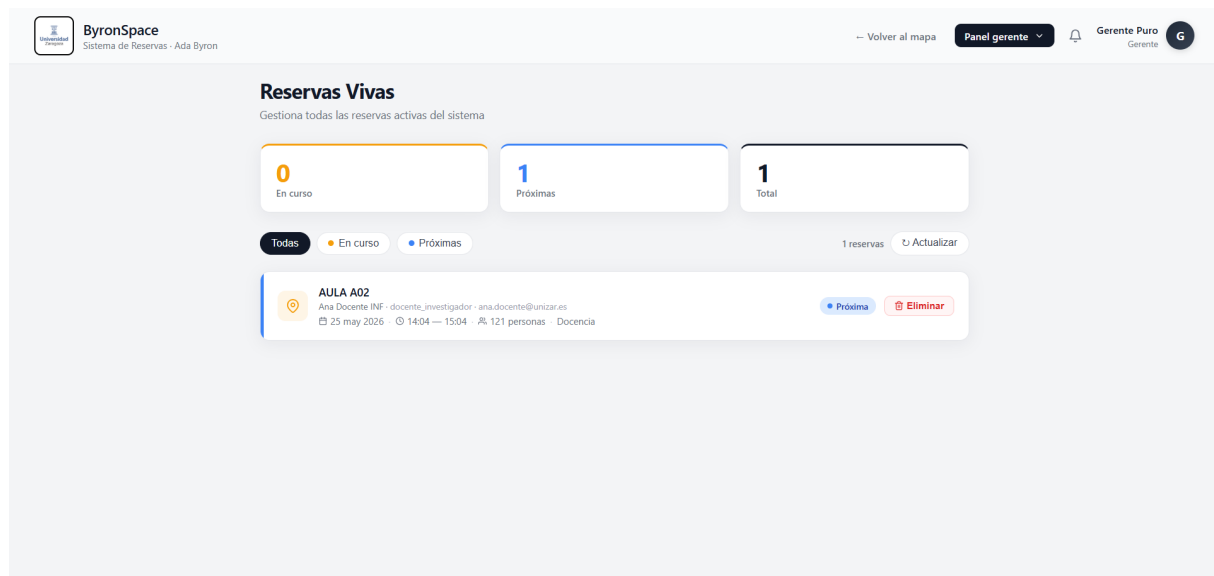


Figura 13: Pantalla del panel de reservas vivas del gerente

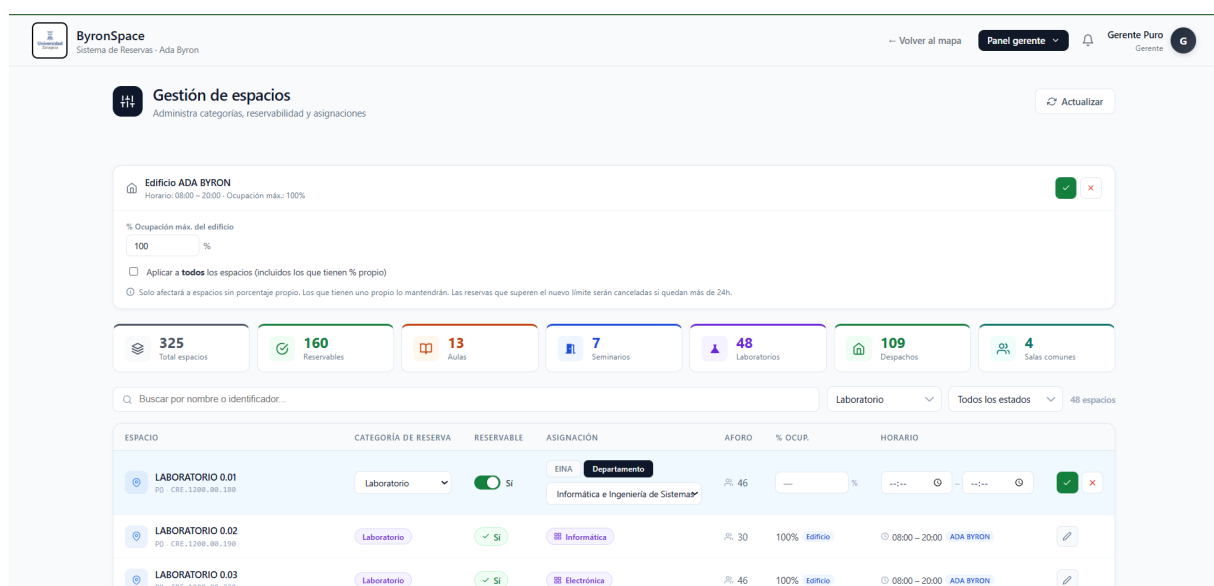


Figura 14: Pantalla del panel de gestión de espacios

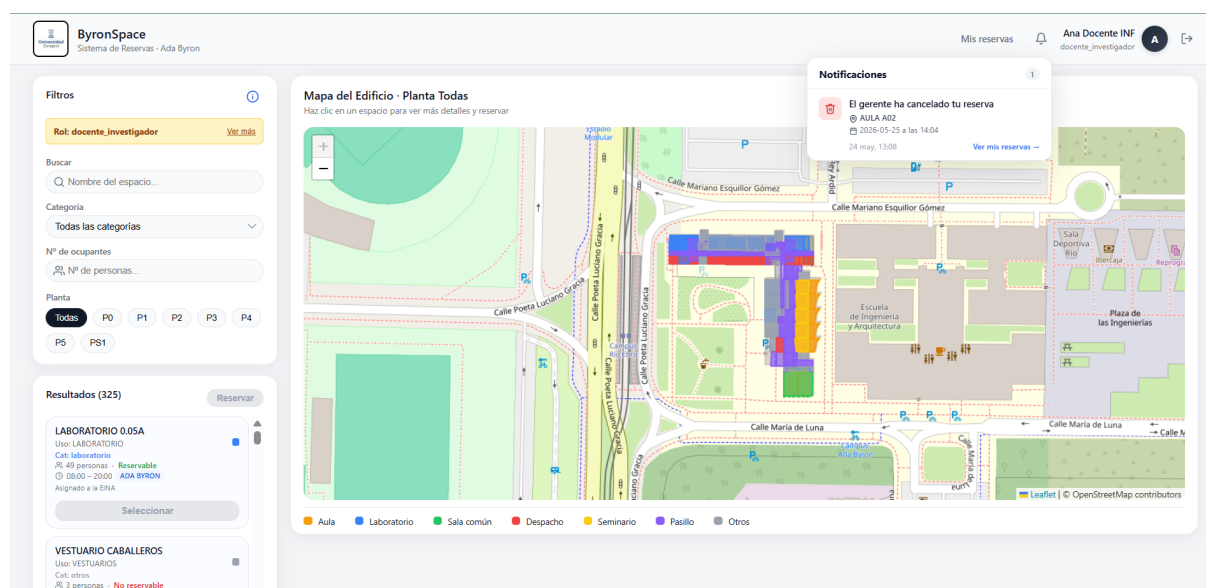


Figura 15: Pantalla del panel de notificaciones

3.1.3. Problemas hallados

Durante el desarrollo del proyecto el equipo se encontró con una serie de dificultades técnicas que conviene mencionar.

Gestión del timezone

El problema más relevante fue la gestión de la zona horaria. El servidor Docker corre en UTC mientras que las reservas se almacenan y validan en hora local española (UTC+2 en verano). Esto provocó errores sutiles en las validaciones de antelación mínima de 24 horas y en el filtrado de reservas vivas, ya que la hora actual obtenida con `new Date()` devolvía la hora UTC en vez de la hora local. El problema se resolvió utilizando `Intl.DateTimeFormat` con la zona horaria `Europe/Madrid` en todos los puntos del código donde se manipula la hora actual.

Comunicación asíncrona con RabbitMQ

La implementación del patrón RPC sobre RabbitMQ para la comunicación entre el gateway y el app-server requirió gestionar correctamente la correlación de mensajes y los timeouts. En las primeras versiones se producían situaciones en las que una respuesta tardía llegaba a la cola equivocada o no se procesaba correctamente. Esto se resolvió generando un `correlationId` único por petición y usando colas de respuesta exclusivas y temporales.

Mapecto entre PostGIS y el modelo de dominio

Los datos geográficos de Ada Byron vienen de PostGIS con nombres de columnas y valores en un formato específico que no coincide directamente con el modelo de dominio. Hubo que realizar un trabajo de mapeo y normalización en los repositorios Sequelize para traducir entre el formato de la base de datos y las entidades del dominio, incluyendo la gestión de tipos inesperados como valores numéricos que llegaban como cadenas de texto.

Propagación de cambios por la arquitectura

La separación estricta entre capas de la arquitectura hexagonal, si bien aporta claridad y orden al código, hace que cualquier cambio tenga que aplicarse en varios sitios a la vez. Modificar una entidad del dominio implica revisar los repositorios, los casos de uso y a veces también los controladores y rutas del gateway. A esto se suma que el gateway y el app-server son dos servicios independientes que se comunican mediante mensajes, por lo que si se cambia el nombre de una acción, los campos de un payload o el formato de una respuesta, hay que actualizarlo en los dos servicios a la vez. Durante el desarrollo esto requirió especial atención para evitar que un cambio en un sitio dejara desactualizado otro.

Ambigüedades en el enunciado

Algunas partes del enunciado presentaban ambigüedades o incompletitudes que se trasladaban directamente al código en forma de inconsistencias. El caso más relevante fue la lógica de asignación de despachos y sus reglas de reserva, donde la combinación de los requisitos O3 y O7 generaba casos límite que no estaban explícitamente contemplados. En todos los casos el equipo resolvió las dudas mediante discusión interna, tomando decisiones razonadas y documentándolas en los requisitos y en el código.

3.1.4. Aspectos de usabilidad

La interfaz de la aplicación ha prestado especial atención a que el usuario comprenda en todo momento qué está pasando y por qué. A continuación se describen los principales aspectos de usabilidad tenidos en cuenta durante el desarrollo.

Mensajes de error descriptivos

Cuando una acción no es posible, el sistema muestra un mensaje concreto que explica el motivo. En el caso de las reservas, el backend devuelve mensajes específicos indicando por ejemplo que el espacio ya está reservado en esa franja horaria mostrando las franjas ocupadas, que el número de ocupantes supera el aforo permitido con el porcentaje de ocupación actual, o que la reserva no cumple la antelación mínima de 24 horas. En el panel de administración del gerente, el sistema también muestra mensajes descriptivos cuando se intenta realizar una operación inválida, como asignar un espacio a un titular incompatible con su categoría de reserva, introducir un horario con la hora de apertura posterior a la de cierre, o indicar un porcentaje de ocupación fuera de rango.

Codificación por colores

Los espacios del mapa se muestran con colores distintos según su categoría de reserva, lo que permite identificar visualmente el tipo de espacio de un vistazo. Cuando un espacio tiene un porcentaje de ocupación inferior al 100 %, el aforo original aparece tachado junto al aforo efectivo, también en el color de su categoría.

Botón de reserva inteligente

El botón de selección para reservar un espacio se deshabilita automáticamente cuando el sistema detecta que la reserva no sería posible por razones estructurales: si el espacio no es reservable, si está asignado a un departamento distinto al del usuario, o si la categoría del espacio no es reservable por el rol del usuario. De esta forma se evita que el usuario inicie un proceso de reserva que el sistema va a rechazar, ahorrándole tiempo y evitando confusiones. No obstante, el servidor de aplicaciones realiza igualmente

todas estas validaciones, de forma que aunque alguien realizara la petición directamente desde un cliente externo como una terminal, el sistema respondería con el error correspondiente.

Tarjetas informativas de espacios

Cada espacio se muestra en una tarjeta con información detallada: nombre, uso físico, categoría de reserva, aforo efectivo teniendo en cuenta el porcentaje de ocupación, tipo de asignación (EINA, departamento o persona) y a quién está asignado en caso de departamento o persona, si es reservable o no, y el horario efectivo del espacio, que será el propio si lo tiene configurado o el del edificio en caso contrario. Esta información permite al usuario evaluar rápidamente si un espacio es adecuado para sus necesidades antes de intentar reservarlo.

Tarjeta informativa del usuario

La interfaz muestra al usuario una tarjeta con su información relevante: rol actual, departamento al que está adscrito si corresponde, y los tipos de espacios que puede reservar según su rol. Esto permite al usuario conocer de antemano sus permisos sin necesidad de intentar reservas que el sistema va a rechazar.

Confirmaciones mediante modales

Las acciones destructivas como cancelar una reserva propia o eliminar una reserva por parte del gerente se confirman mediante modales en vez de alertas nativas del navegador, lo que mejora la experiencia tanto en escritorio como en dispositivos móviles.

Indicador de notificaciones

El panel de notificaciones muestra un indicador numérico en rojo cuando hay notificaciones no leídas, permitiendo al usuario saber de un vistazo si tiene avisos pendientes sin necesidad de entrar al panel.

Sincronización entre mapa y resultados

Al hacer clic sobre un espacio en el mapa se muestra un popup con la información más relevante del espacio y la sección de resultados realiza un desplazamiento automático hasta la tarjeta correspondiente, que se ilumina para facilitar su identificación. El comportamiento es bidireccional: si el usuario hace clic en una tarjeta de la sección de resultados, el mapa abre automáticamente el popup del espacio correspondiente.

3.2. Arquitectura del sistema

3.2.1. Vista de módulos

El servidor de aplicaciones sigue una arquitectura hexagonal organizada en tres capas bien diferenciadas: *infrastructure*, *application* y *domain*. Esta separación garantiza que la lógica de negocio del dominio sea completamente independiente de los detalles técnicos de infraestructura, facilitando el mantenimiento, la testabilidad y la evolución del sistema.

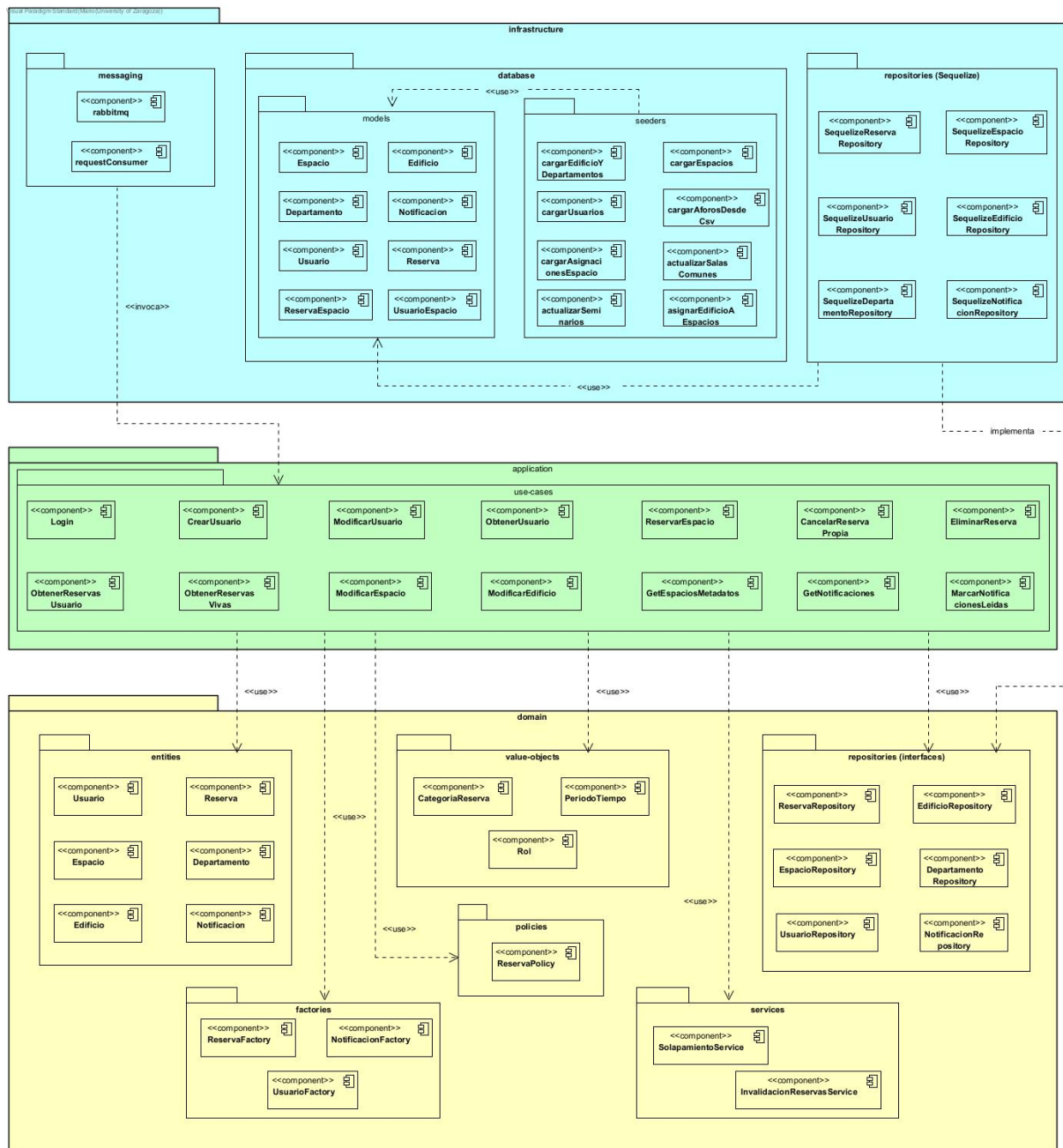


Figura 16: Vista de módulos

La capa de dominio (domain) es el núcleo del sistema y no depende de ninguna otra capa. Contiene seis subpaquetes: **entities** con las seis entidades del sistema (Reserva, Espacio, Usuario, Edificio, Departamento y Notificacion), **value-objects** con los tres objetos valor (CategoriaReserva, PeriodoTiempo y Rol), **factories** con las factorías de creación de los agregados Reserva, Notificacion y Usuario, **repositories** con las interfaces abstractas de acceso a datos, **services** con los servicios de dominio y **policies** con la política de reservas. Esta capa no depende de ninguna otra capa del sistema.

La capa de aplicación (application) contiene un único subpaquete **use-cases** con los catorce casos de uso del sistema. Los casos de uso orquestan la lógica de negocio usando las entidades, objetos valor, factorías, servicios, políticas e interfaces de repositorio.

del dominio. No conocen los detalles de infraestructura, trabajan exclusivamente con las interfaces abstractas definidas en el dominio.

La capa de infraestructura (infrastructure) contiene tres subpaquetes: messaging con los adaptadores de entrada RabbitMQ que reciben los mensajes del gateway e invocan los casos de uso correspondientes, repositories con las implementaciones Sequelize de las interfaces de repositorio definidas en el dominio, y database con los modelos Sequelize y los seeders de carga inicial de datos.

El flujo de dependencias respeta estrictamente la regla de dependencia de la arquitectura hexagonal: infrastructure depende de application y domain, application depende de domain, y domain no depende de nadie. Las implementaciones de repositorio de infraestructura implementan las interfaces abstractas del dominio, lo que permite sustituir la tecnología de persistencia sin modificar ni el dominio ni los casos de uso.

Cabe mencionar algunas decisiones de nomenclatura tomadas durante el desarrollo. El subpaquete de casos de uso se denomina use-cases en vez de services porque, aunque los casos de uso son técnicamente servicios de la capa de aplicación y están en un módulo separado del resto, el término services ya está reservado en DDD para los servicios de dominio, que tienen una semántica diferente. Usar use-cases hace explícita la intención y evita ambigüedades incluso estando en capas distintas.

Respecto a la mezcla de nombres en inglés y español en los casos de uso, el proyecto comenzó siguiendo la convención de nombrar la acción en inglés y el concepto de dominio en español, resultando en nombres como GetNotificaciones o GetEspaciosMetadatos. Con el tiempo el equipo evolucionó hacia usar español de forma consistente en todo, resultando en nombres como ObtenerReservasVivas o CrearUsuario. Cuando se detectó la inconsistencia se intentó unificar todos los nombres, pero al estar directamente acoplados a las acciones de la capa de mensajería el cambio habría requerido modificar en todos los sitios: el gateway, el consumidor de mensajes y los tests de integración. Ante el riesgo de introducir errores se decidió documentar la inconsistencia y mantener los nombres originales.

3.2.2. Vista de componentes y conectores

El sistema sigue una arquitectura N-Tier de 4 niveles, como se aprecia en el diagrama de componentes y conectores. Cada nivel agrupa los componentes con responsabilidades similares y la comunicación entre niveles sigue la restricción fundamental de esta arquitectura: un nivel solo puede comunicarse con sus vecinos inmediatos, nunca saltándose niveles.

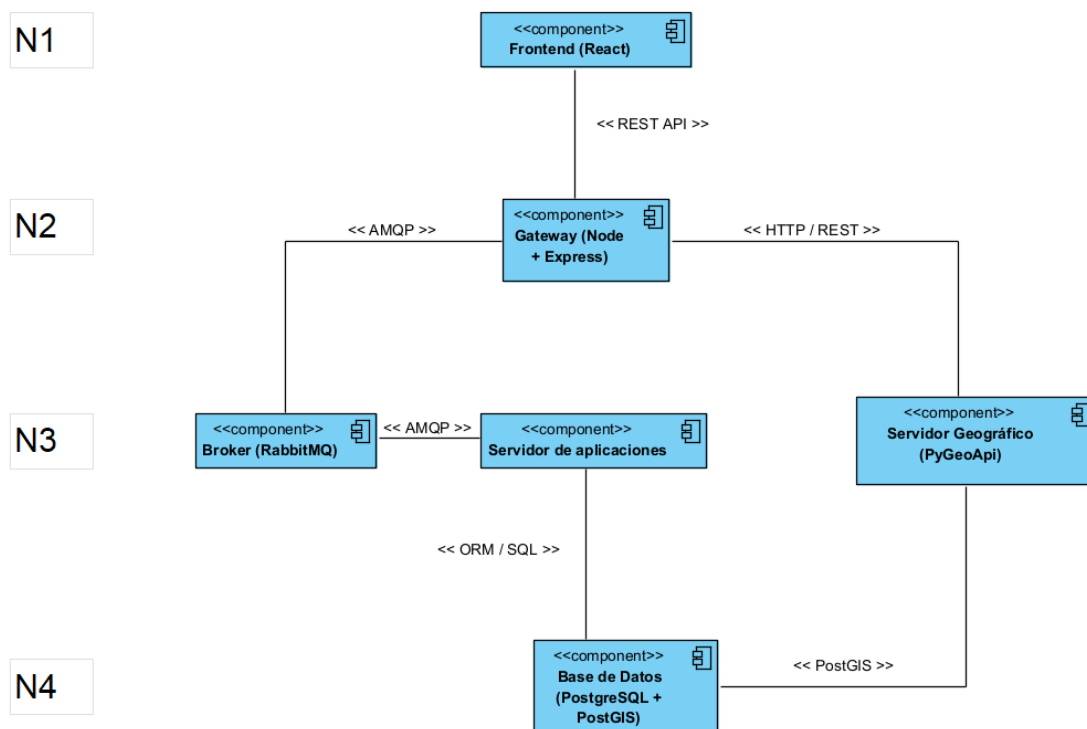


Figura 17: Vista de componentes y conectores

El nivel 1 (N1) está formado por el frontend desarrollado en React. Es la capa de presentación con la que interactúa el usuario directamente a través del navegador. Se comunica exclusivamente con el nivel 2 mediante llamadas HTTP a la API REST del gateway.

El nivel 2 (N2) está formado por el gateway, implementado con Node.js y Express. Actúa como punto de entrada único del sistema, recibiendo todas las peticiones del frontend, gestionando la autenticación mediante JWT y enrutando cada petición hacia el componente del nivel 3 correspondiente. La comunicación con el servidor de aplicaciones se produce a través del broker de mensajes RabbitMQ mediante el protocolo AMQP, mientras que la comunicación con el servidor geográfico se realiza mediante HTTP/REST.

El nivel 3 (N3) está formado por tres componentes: el broker de mensajes RabbitMQ, el servidor de aplicaciones y el servidor geográfico PyGeoAPI. RabbitMQ actúa como intermediario entre el gateway y el servidor de aplicaciones, implementando el patrón RPC sobre AMQP que permite una comunicación asíncrona desacoplada entre ambos servicios. El servidor de aplicaciones contiene toda la lógica de negocio del sistema y se comunica con la base de datos del nivel 4 mediante Sequelize como ORM. PyGeoAPI expone los datos geográficos de los espacios del edificio Ada Byron directamente desde la base de datos PostGIS del nivel 4.

El nivel 4 (N4) está formado por la base de datos PostgreSQL con la extensión PostGIS, que almacena tanto los datos relacionales del sistema como los datos geográficos de los espacios del edificio.

La comunicación asíncrona mediante mensajes entre el gateway (N2) y el servidor de aplicaciones (N3) es uno de los aspectos más destacados de la arquitectura y uno de los requisitos para alcanzar el nivel de sobresaliente. Para implementarla se ha utilizado RabbitMQ como broker de mensajes, AMQP como protocolo de comunicación y el patrón RPC sobre mensajería asíncrona.

RabbitMQ es un broker de mensajes que actúa como intermediario entre dos servicios, permitiendo que se comuniquen sin necesidad de conocerse directamente ni de estar disponibles al mismo tiempo. AMQP (Advanced Message Queuing Protocol) es el protocolo estándar que define cómo se envían, enrutan y consumen los mensajes a través del broker. Se ha elegido RabbitMQ por su fiabilidad, su amplio soporte en Node.js y porque permite implementar el patrón RPC de forma sencilla.

El patrón RPC (Remote Procedure Call) sobre mensajería permite al gateway invocar operaciones en el servidor de aplicaciones como si fueran llamadas locales, pero de forma desacoplada. El gateway actúa como productor: envía un mensaje a la cola indicando la acción a ejecutar y el payload correspondiente, e incluye un `correlationId` único y el nombre de una cola temporal exclusiva donde espera la respuesta. El servidor de aplicaciones actúa como consumidor: recibe el mensaje, ejecuta el caso de uso correspondiente y devuelve la respuesta a la cola indicada. Cuando el gateway recibe la respuesta con el `correlationId` correcto, la procesa y la devuelve al frontend si corresponde. Este diseño desacopla completamente ambos servicios y permite que escalen de forma independiente.

Los dos flujos principales de comunicación del sistema son: el flujo de datos geográficos, que va desde el frontend hasta PyGeoAPI pasando por el gateway y termina en la base de datos PostGIS, devolviendo los datos geográficos al cliente; y el flujo de lógica de negocio, que va desde el frontend hasta el servidor de aplicaciones pasando por el gateway y RabbitMQ, accediendo a la base de datos cuando es necesario. En este segundo flujo, la respuesta viaja de vuelta hasta el cliente cuando la operación lo requiere, como en el caso de una consulta de reservas o de espacios, mientras que otras operaciones simplemente devuelven una confirmación. El papel de PyGeoAPI y el flujo de datos geográficos se describen con más detalle en la sección 3.4, dedicada a la implementación del sistema de información geográfica.

3.2.3. Vista de despliegue

La vista de despliegue muestra cómo se distribuyen los componentes del sistema en tiempo de ejecución. El sistema se despliega mediante Docker Compose, que orquesta cinco contenedores dentro de una red interna llamada `reservas_net`. Esta red aísla los servicios del exterior y permite que se comuniquen entre sí usando los nombres de los contenedores como hostnames.

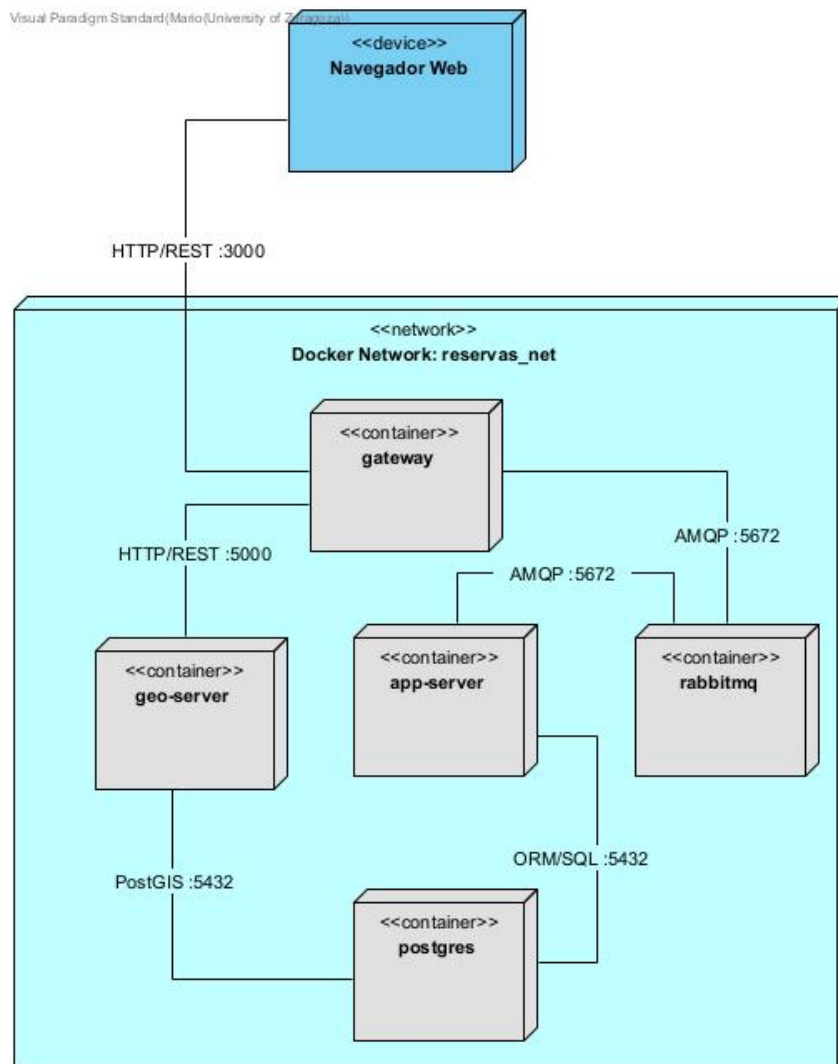


Figura 18: Vista de despliegue — Contenedores Docker

El contenedor gateway es el único punto de entrada al sistema desde el exterior. Expone el puerto 3000 y recibe todas las peticiones HTTP/REST del navegador web del cliente. Desde ahí enruta las peticiones hacia los servicios del nivel 3: hacia rabbitmq mediante AMQP para la lógica de negocio, y hacia geo-server mediante HTTP/REST para los datos geográficos.

El contenedor rabbitmq actúa como broker de mensajes entre el gateway y el servidor de aplicaciones. Expone el puerto 5672 para la comunicación AMQP y el puerto 15672 para su panel de administración web.

El contenedor app-server contiene el servidor de aplicaciones con la arquitectura hexagonal. Recibe los mensajes del gateway a través de RabbitMQ, ejecuta los casos de uso correspondientes y accede a la base de datos mediante Sequelize a través del puerto 5432.

El contenedor geo-server ejecuta PyGeoAPI y expone los datos geográficos de los espacios del edificio Ada Byron mediante el estándar OGC API Features. Accede directamente a la base de datos PostGIS a través del puerto 5432.

El contenedor postgres ejecuta PostgreSQL con la extensión PostGIS y almacena tanto los datos relacionales del sistema como los datos geográficos. Utiliza un volumen Docker llamado pgdata para persistir los datos entre reinicios del sistema.

3.3. Diseño Dirigido por el Dominio

3.3.1. Diagrama de clases de la capa de dominio

El diagrama de clases de la capa de dominio tiene como objetivo representar de forma visual el modelo de dominio del sistema, mostrando las entidades, objetos valor, factorías, servicios, política y repositorios que lo componen, así como las relaciones y dependencias entre ellos. Este diagrama permite comprender la estructura del dominio de forma independiente a los detalles de implementación. Con el fin de mantener el diagrama legible, se han omitido algunos atributos de menor relevancia para el dominio.

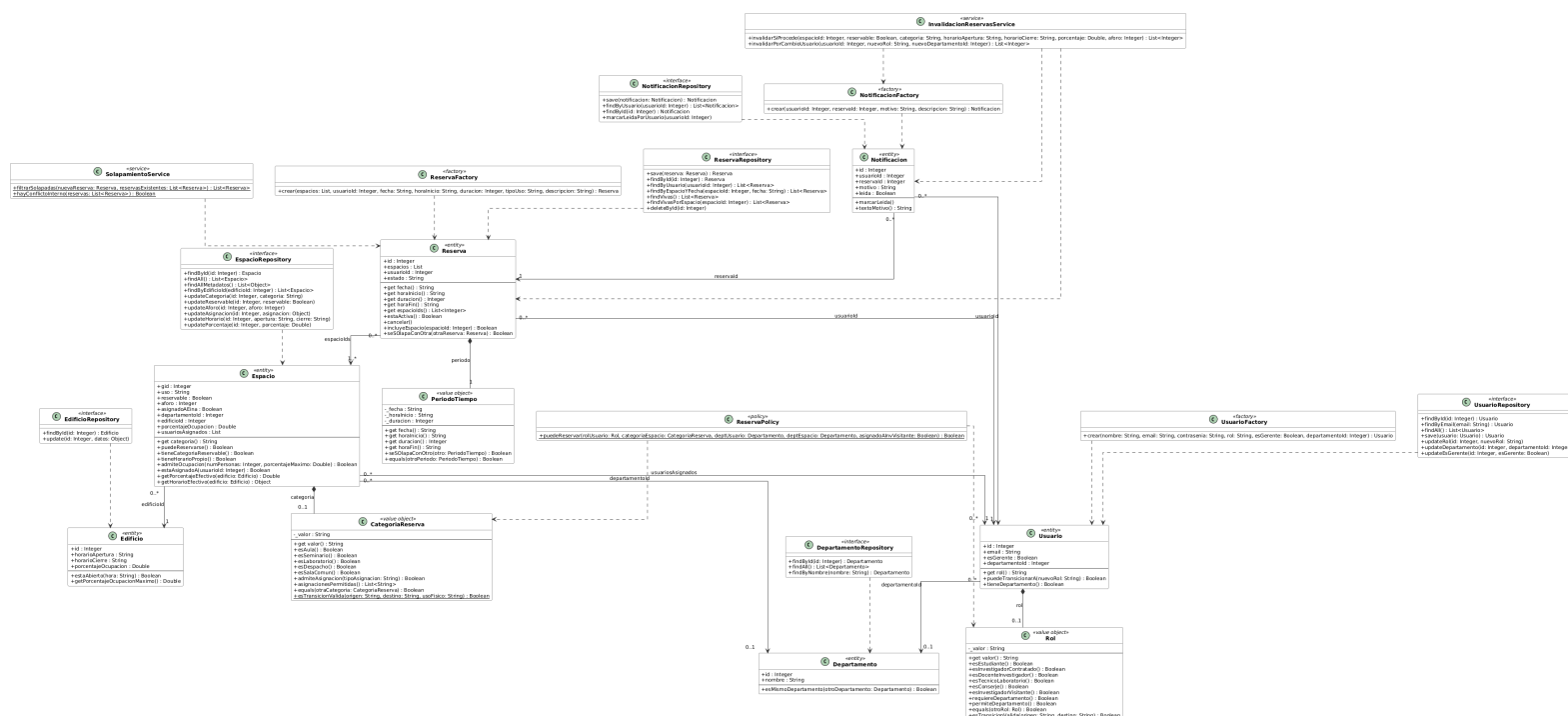


Figura 19: Diagrama de clases de la capa de dominio

Debido a la cantidad de clases y relaciones del diagrama, su visualización en el documento puede resultar limitada a simple vista, aunque haciendo zoom sobre el documento se puede consultar con claridad. Para verlo en su tamaño original se recomienda acceder al repositorio de documentación del proyecto, donde se encuentran disponibles tanto la memoria en PDF como todos los diagramas incluidos en ella: <https://github.com/UNIZAR-30249-2026-RESERVASADA/docs>

Las entidades principales del sistema son Reserva, Espacio y Usuario, que constituyen las raíces de sus respectivos agregados. Cada una de ellas encapsula objetos valor: Reserva contiene un PeriodoTiempo que representa el intervalo temporal de la reserva,

Espacio contiene una `CategoriaReserva` que representa el tipo de espacio bajo el que puede ser reservado, y `Usuario` contiene un `RoI` que representa sus permisos en el sistema. Las entidades secundarias `Edificio`, `Departamento` y `Notificacion` completan el modelo de dominio.

Las interfaces de repositorio definen el contrato de acceso a datos que la capa de infraestructura debe implementar. La política `ReservaPolicy` centraliza las reglas que determinan qué roles pueden reservar qué tipos de espacios. Las factorías `ReservaFactory`, `NotificacionFactory` y `UsuarioFactory` encapsulan la creación de las entidades raíz de cada agregado.

Los servicios de dominio encapsulan lógica que no pertenece a ninguna entidad concreta. `SolapamientoService` se encarga de detectar conflictos de horario entre reservas, e `InvalidacionReservasService` gestiona la cancelación automática de reservas cuando se produce un cambio estructural en el sistema. En los siguientes apartados se describen en detalle cada uno de los elementos del modelo de dominio, las decisiones de diseño tomadas y los patrones aplicados.

3.3.2. Entidades, objetos valor y agregados

Entidades

Las entidades son objetos del dominio con identidad propia, distinguidos por un identificador único. En el sistema las entidades son `Reserva`, `Espacio`, `Usuario`, `Edificio`, `Departamento` y `Notificacion`.

`Reserva`, `Espacio` y `Usuario` son entidades porque tienen un ciclo de vida propio y su estado puede cambiar a lo largo del tiempo. `Edificio` es una entidad porque representa un elemento físico único del sistema.

`Departamento` y `Notificacion` merecen una mención especial ya que podrían haberse modelado como objetos valor. `Departamento` tiene un identificador asignado por la base de datos y es referenciado desde otros agregados por ese id. Si fuera un objeto valor, dos departamentos con el mismo nombre serían considerados el mismo departamento, lo cual no es correcto. `Notificacion` tiene un ciclo de vida propio: nace como no leída y puede pasar a leída mediante el método `marcarLeida()`. Al tener estado mutable no puede ser un objeto valor.

Objetos valor

Los objetos valor son objetos del dominio sin identidad propia, dos instancias con los mismos valores son consideradas iguales y son inmutables. En el sistema los objetos valor son `PeriodoTiempo`, `CategoriaReserva` y `Rol`.

`PeriodoTiempo` es un objeto valor porque un intervalo de tiempo se define completamente por su fecha, hora de inicio y duración. `CategoriaReserva` es un objeto valor porque la categoría de un espacio es simplemente un valor que lo clasifica, sin identidad propia. `Rol` es un objeto valor porque el rol de un usuario es un valor del dominio que define sus permisos, sin ciclo de vida propio. Los tres implementan el método `equals()` para comparar por valor y garantizan la inmutabilidad mediante `Object.freeze()` al final del constructor.

Agregados

Un agregado es una unidad de consistencia del dominio formada por una entidad raíz y, opcionalmente, objetos valor que pertenecen a ella. En este sistema todos los agregados tienen una única entidad raíz.

El sistema tiene cinco agregados. El agregado Reserva contiene como objeto valor PeriodoTiempo, el agregado Espacio contiene CategoriaReserva, y el agregado Usuario contiene Rol. Los agregados Edificio, Departamento y Notificacion tienen únicamente su entidad raíz.

Las referencias entre agregados se realizan siempre de forma indirecta mediante identificadores, nunca mediante referencias directas a objetos. Existe sin embargo una excepción consciente en el agregado Espacio: el atributo usuariosAsignados guarda objetos con la forma { id, rol, departamentoId } en vez de solo los identificadores. El motivo es que ReservaPolicy necesita saber el rol y el departamento de cada usuario asignado para decidir si una reserva es válida, y si solo se guardaran los ids habría que ir a buscar cada usuario a la base de datos en cada validación. Para evitar esas consultas extra los datos se guardan directamente en el espacio y se mantienen actualizados desde el caso de uso ModificarEspacio.

3.3.3. Factorías y repositorios

Factorías

Las factorías encapsulan la creación de las entidades raíz de cada agregado, asegurando que cualquier instancia creada cumple todas las precondiciones del dominio desde el primer momento. El sistema tiene tres factorías. ReservaFactory crea instancias de Reserva con estado inicial "aceptada", ya que una reserva recién creada siempre nace en ese estado y nunca se crea directamente como cancelada o finalizada. NotificacionFactory crea instancias de Notificacion con leida=false y fechaCreacion igual al momento actual, garantizando que las notificaciones siempre nacen como no leídas y con la fecha correcta sin que el código que las crea tenga que preocuparse de ello. UsuarioFactory crea instancias de Usuario sin id ya que el identificador lo asigna la base de datos al persistir, no el dominio. Las tres factorías reciben solo los parámetros esenciales para construir una instancia válida, delegando las validaciones de invariantes en los propios constructores de las entidades.

Repositorios

Los repositorios son interfaces abstractas del dominio que definen el contrato de acceso a datos que la capa de infraestructura debe implementar. Los casos de uso dependen de estas interfaces y no de las implementaciones concretas de Sequelize, de forma que si en algún momento se quisiera cambiar la tecnología de persistencia no habría que tocar ni el dominio ni los casos de uso. Cada agregado tiene su propio repositorio: ReservaRepository, EspacioRepository, UsuarioRepository, EdificioRepository, DepartamentoRepository y NotificacionRepository. Los métodos de cada repositorio siguen el patrón de interfaz reveladora, sus nombres dejan claro qué hacen con solo leerlos, sin necesidad de consultar la implementación. Ejemplos de esto son findVivas(), findVivasPorEspacio() o marcarLeidaPorUsuario().

3.3.4. Servicios, interfaces reveladoras, aserciones y funciones sin efectos secundarios

Servicios

Los servicios de dominio encapsulan lógica que no pertenece a ninguna entidad concreta. El sistema tiene dos servicios de dominio.

`SolapamientoService` detecta conflictos de horario entre reservas. La entidad `Reserva` tiene el método `seSolapaConOtra()` para compararse con otra reserva individual, pero la lógica de detectar conflictos dentro de un conjunto de reservas no es responsabilidad de ninguna reserva concreta sino de algo externo que tiene acceso a todas ellas a la vez. Se ha implementado como servicio de dominio porque ninguna entidad individual debería tener la responsabilidad de gestionar colecciones de reservas ajenas.

`InvalidacionReservasService` gestiona la cancelación automática de reservas cuando se produce un cambio estructural en el sistema. Se ha implementado como servicio de dominio porque la invalidación implica coordinar varias cosas a la vez: consultar reservas, evaluar la política de reservas, cancelar las afectadas y generar notificaciones. Todo ese trabajo conjunto no tiene sentido meterlo en una sola entidad, ya que ninguna de ellas tiene por qué saber lo que hacen las demás.

Interfaces reveladoras

Una interfaz reveladora es aquella cuyos nombres de métodos expresan directamente su significado en el dominio sin necesidad de leer su implementación. En el sistema se ha aplicado este patrón de forma consistente en todas las entidades, objetos valor, servicios y repositorios.

Ejemplos en entidades: `puedeReservarse()`, `estaActiva()`, `tieneCategoriaReservable()`, `tieneHorarioPropio()`, `estaAsignadoA()`, `esMismoDepartamento()`. Ejemplos en objetos valor: `esAula()`, `esSeminario()`, `esDespacho()`, `esEstudiante()`, `esInvestigadorVisitante()`, `requiereDepartamento()`. Ejemplos en repositorios: `findVivas()`, `findVivasPorEspacio()`, `marcarLeidaPorUsuario()`.

Aserciones

Las aserciones son comprobaciones que verifican que el objeto ha quedado en un estado válido tras su construcción. Se implementan mediante `console.assert()` al final del constructor de cada entidad y objeto valor, justo después de asignar los atributos. A diferencia de las precondiciones, que lanzan excepciones cuando los datos de entrada son inválidos, las aserciones comprueban que el estado interno del objeto es consistente tras la construcción.

Por ejemplo, en la entidad `Reserva` se comprueba que `espacios` tiene al menos un elemento, que `usuarioId` no es null y que `estado` es uno de los valores válidos. En el objeto valor `PeriodoTiempo` se comprueba que `duracion` es positiva y que `fecha` y `horaInicio` tienen el formato correcto. En la entidad `Notificacion` se comprueba tras el método `marcarLeida()` que `leida` es efectivamente true.

Funciones sin efectos secundarios

Una función sin efectos secundarios es aquella que solo lee información y devuelve un resultado sin modificar ningún estado. En el sistema se ha aplicado este principio

de forma consistente en todos los métodos de consulta de entidades, objetos valor y servicios.

Todos los métodos de los objetos valor siguen este principio por definición, ya que son inmutables. En las entidades, los métodos de consulta como `estaActiva()`, `incluyeEspacio()`, `admiteOcupacion()` o `estaAsignadoA()` solo leen el estado del objeto sin modificarlo. Los métodos de `SolapamientoService` también siguen este principio, dado un conjunto de reservas siempre devuelven el mismo resultado. Los únicos métodos que modifican estado son los que lo hacen de forma intencionada, como `cancelar()` en `Reserva` o `marcarLeida()` en `Notificacion`, y están claramente documentados como tales.

3.3.5. Decisiones de diseño relevantes

En el servidor de aplicaciones no se han implementado transacciones atómicas a nivel de caso de uso. El equipo tenía previsto implementarlas pero no dio tiempo antes de la entrega. `Sequelize` las soporta mediante `sequelize.transaction()`, por lo que su implementación futura sería sencilla. En la práctica el impacto es limitado ya que cada caso de uso sigue un orden correcto de operaciones y los fallos a mitad de una operación son poco frecuentes en condiciones normales de uso.

3.4. Implementación del SIG

El sistema incorpora un Sistema de Información Geográfica (SIG) para representar los espacios del edificio Ada Byron sobre un mapa interactivo. La implementación se apoya en tres componentes principales: `PostGIS`, `PyGeoAPI` y `Leaflet`.

Los datos geográficos de los espacios se almacenan en `PostgreSQL` mediante la extensión `PostGIS`. Cada espacio tiene un campo `geom` de tipo geometría que almacena su forma y posición geográfica. `PyGeoAPI` se conecta directamente a la tabla `espacios` de la base de datos y devuelve los datos en formato `GeoJSON`, que es un formato estándar para representar datos geográficos en `JSON`.

El frontend nunca accede directamente a `PyGeoAPI` sino siempre a través del gateway, que actúa como proxy entre ambos. Esto mantiene la coherencia con la arquitectura N-Tier y evita exponer `PyGeoAPI` al exterior. Cuando `PyGeoAPI` pagina los resultados incluye en la respuesta enlaces para navegar entre páginas que apuntan directamente a su propia dirección. El gateway intercepta esas respuestas y reescribe esos enlaces para que apunten a sus propias rutas, de forma que el frontend siempre habla con el gateway sin saber que `PyGeoAPI` existe.

En el frontend los datos `GeoJSON` devueltos por el gateway se renderizan sobre un mapa interactivo mediante `Leaflet`. El mapa base es de tipo `WMS` obtenido desde `OpenStreetMap`, sobre el que se superponen los polígonos de los espacios del edificio. Cada espacio se colorea según su categoría de reserva, permitiendo identificarlos visualmente de un vistazo. Al hacer clic sobre un espacio en el mapa se muestra un popup con su información y se sincroniza con la sección de resultados de la interfaz.

3.5. Documentación de la API

La API REST del gateway es el único punto de entrada al sistema desde el exterior. Todas las rutas protegidas requieren un token JWT en la cabecera `Authorization: Bearer <token>`. La documentación interactiva de la API está disponible en `http://localhost:3000/docs/` cuando el backend está en marcha, generada automáticamente mediante Swagger. A continuación se documentan todos los endpoints disponibles.

Autenticación

POST /api/auth/login	
Descripción	Inicia sesión con email y contraseña. Devuelve un token JWT y los datos del usuario.
Autenticación	No requerida
Body	email (string, obligatorio), password (string, obligatorio)
Respuesta 200	token (string), usuario {id, nombre, email, rol, esGerente, departamentoId}
Errores	400 campos inválidos o ausentes, 401 contraseña incorrecta, 404 usuario no encontrado

Espacios

GET /api/espacios/metadatos	
Descripción	Devuelve los metadatos de todos los espacios del sistema.
Autenticación	No requerida
Respuesta 200	Lista de espacios con sus metadatos: id, nombre, categoría, aforo, reservable, asignación y horario.
Errores	500 error interno del servidor

PATCH /api/espacios/:id	
Descripción	Modifica las propiedades de un espacio.
Autenticación	Token JWT. Solo gerentes.
Path params	id (integer): identificador del espacio
Body	reservable (boolean), categoria (string), aforo (integer), asignadoAEina (boolean), departamentoId (integer), usuariosAsignados (array), horarioApertura (string), horarioCierre (string), porcentajeOcupacion (number). Todos opcionales.
Respuesta 200	Espacio actualizado con sus nuevos datos y lista de reservas canceladas.
Errores	400 datos inválidos, 401 sin token, 403 no es gerente, 404 espacio no encontrado

Edificio

PATCH /api/edificio/:id	
Descripción	Modifica el horario y/o porcentaje de ocupación del edificio.
Autenticación	Token JWT. Solo gerentes.
Path params	id (integer): identificador del edificio
Query params	afectarTodos (boolean): si true aplica el cambio de porcentaje a todos los espacios, si false solo a los que heredan el valor del edificio
Body	horarioApertura (string), horarioCierre (string), porcentajeOcupacion (number). Todos opcionales.
Respuesta 200	Edificio actualizado con sus nuevos datos y lista de reservas canceladas.
Errores	400 datos inválidos, 401 sin token, 403 no es gerente, 404 edificio no encontrado

Reservas

POST /api/reservas	
Descripción	Crea una nueva reserva.
Autenticación	Token JWT
Body	espacios (array de {espacioId, numPersonas}, obligatorio), fecha (string YYYY-MM-DD, obligatorio), horaInicio (string HH:MM, obligatorio), duracion (integer en minutos, obligatorio), tipoUso (string, opcional), descripcion (string, opcional)
Respuesta 201	Reserva creada con id y estado.
Errores	400 datos inválidos o reserva no permitida, 401 sin token, 403 sin permisos para reservar ese espacio

GET /api/reservas/mis-reservas	
Descripción	Devuelve todas las reservas del usuario autenticado.
Autenticación	Token JWT
Respuesta 200	Lista de reservas del usuario.
Errores	401 sin token

GET /api/reservas/vivas	
Descripción	Devuelve todas las reservas activas del sistema.
Autenticación	Token JWT. Solo gerentes.
Respuesta 200	Lista de todas las reservas activas con datos del usuario que las realizó.
Errores	401 sin token, 403 no es gerente

DELETE /api/reservas/:id	
Descripción	Cancela una reserva propia del usuario autenticado.
Autenticación	Token JWT
Path params	id (integer): identificador de la reserva
Respuesta 200	Reserva cancelada con id y estado.
Errores	400 reserva no activa, 401 sin token, 403 la reserva no pertenece al usuario, 404 reserva no encontrada

DELETE /api/reservas/:id/admin	
Descripción	Cancela cualquier reserva del sistema y notifica al usuario que la realizó.
Autenticación	Token JWT. Solo gerentes.
Path params	id (integer): identificador de la reserva
Respuesta 200	Confirmación de cancelación.
Errores	400 reserva en curso o menos de 24h de antelación, 401 sin token, 403 no es gerente, 404 reserva no encontrada

Usuarios

GET /api/usuarios/me	
Descripción	Devuelve los datos del usuario autenticado.
Autenticación	Token JWT
Respuesta 200	{id, nombre, email, rol, esGerente, departamentoId}
Errores	401 sin token, 404 usuario no encontrado

POST /api/usuarios	
Descripción	Crea un nuevo usuario en el sistema.
Autenticación	Token JWT. Solo gerentes.
Body	nombre (string, obligatorio), email (string, obligatorio), contrasenia (string, obligatorio), rol (string, opcional), departamentoId (integer, opcional), esGerente (boolean, opcional)
Respuesta 201	Usuario creado con sus datos.
Errores	400 datos inválidos, 401 sin token, 403 no es gerente, 409 email ya existente

PATCH /api/usuarios/:id	
Descripción	Modifica el rol, departamento y/o flag de gerente de un usuario.
Autenticación	Token JWT. Solo gerentes.
Path params	id (integer): identificador del usuario
Body	rol (string, opcional), departamentoId (integer, opcional), esGerente (boolean, opcional)
Respuesta 200	Usuario actualizado con sus nuevos datos y lista de reservas canceladas.
Errores	400 datos inválidos o transición de rol no permitida, 401 sin token, 403 no es gerente, 404 usuario no encontrado

Notificaciones

GET /api/notificaciones	
Descripción	Devuelve todas las notificaciones del usuario autenticado.
Autenticación	Token JWT
Respuesta 200	Lista de notificaciones con id, motivo, descripcion, leida y fecha-Creacion.
Errores	401 sin token

PATCH /api/notificaciones/leidas	
Descripción	Marca todas las notificaciones del usuario autenticado como leídas.
Autenticación	Token JWT
Respuesta 200	{ok: true}
Errores	401 sin token

Geográfico

GET /api/geo/espacios	
Descripción	Devuelve los datos geográficos de los espacios del edificio Ada Byron en formato GeoJSON. Actúa como proxy hacia PyGeoAPI.
Autenticación	No requerida
Query params	limit (integer, por defecto 1000), offset (integer, opcional)
Respuesta 200	GeoJSON con los espacios del edificio.
Errores	500 error al conectar con PyGeoAPI

3.6. Tests y cobertura

3.6.1. Estrategia de testing

La estrategia de testing del sistema se divide en dos niveles bien diferenciados. Por un lado, tests unitarios en el app-server que verifican la lógica de dominio y los casos de uso de forma aislada. Por otro, tests de integración en el gateway que verifican que las rutas HTTP responden correctamente, que los middlewares de autenticación funcionan bien y que los errores del app-server se propagan con el código HTTP correcto.

En ambos casos se usa Jest como framework de testing. Para los tests de integración del gateway se usa además supertest, una librería que permite lanzar peticiones HTTP reales contra la aplicación Express sin necesidad de levantar un servidor en un puerto real, lo que facilita enormemente la escritura de tests de integración. Los repositorios y el cliente RabbitMQ se mockean en todos los tests para que cada test sea independiente de la infraestructura real y pueda ejecutarse sin base de datos ni RabbitMQ en marcha. La cobertura se mide ejecutando el comando `npm run test:coverage` en cada uno.

3.6.2. Tests unitarios del app-server

Los tests unitarios del app-server están organizados en dos carpetas dentro de `tests/unit/`: `domain/` para los tests del dominio y `application/` para los tests de los casos de uso.

Los tests de dominio cubren las entidades Reserva y Espacio, los tres objetos valor, los dos servicios de dominio y la política de reservas. Las entidades Departamento y Edificio no tienen tests unitarios directos ya que su lógica es sencilla y quedan cubiertas indirectamente a través de los tests de los casos de uso que las utilizan. Cada test comprueba tanto los casos válidos como los inválidos, verificando que los constructores lanzan errores con datos incorrectos, que los métodos devuelven los resultados esperados y que las invariantes se respetan.

Los tests de casos de uso cubren Login, CrearUsuario, ModificarUsuario, ObtenerUsuario, CancelarReservaPropia, EliminarReserva, ModificarEdificio, ModificarEspacio y ReservarEspacio. Cada caso de uso se testa con mocks de los repositorios, organizando los tests en grupos por tipo de validación: casos válidos, validaciones de permisos, validaciones de campos obligatorios y validaciones de negocio. Los casos de uso GetEspaciosMetadatos, GetNotificaciones, MarcarNotificacionesLeidas, ObtenerReservaUsuario y ObtenerReservasVivas no tienen tests unitarios propios. La parte del gateway que los invoca sí está testeada mediante los tests de integración.

En total el app-server cuenta con **453 tests unitarios** pasando en 17 suites.

3.6.3. Tests de integración del gateway

Los tests de integración del gateway usan supertest para lanzar peticiones HTTP reales contra la aplicación Express, con el cliente del app-server mockeado mediante Jest. Esto permite verificar el comportamiento completo del gateway: rutas, middlewares, DTOs y controladores, sin necesidad de que el app-server esté en marcha.

Los tests cubren todas las rutas del gateway: autenticación, reservas, espacios, edificio, notificaciones y usuarios. Para cada ruta se comprueban los casos válidos, la autenticación con token JWT, los permisos de gerente y la propagación correcta de los errores devueltos por el app-server.

En total el gateway cuenta con **58 tests de integración** pasando en 7 suites.

3.6.4. Cobertura

La cobertura del app-server alcanza un **79.32 % de sentencias, 70.98 % de ramas, 76.82 % de funciones y 80.25 % de líneas**. La cobertura es especialmente alta en la capa de dominio — los objetos valor alcanzan un 95.57 %, los servicios un 97.32 % y la política

un 93.93 %. Los casos de uso con menor cobertura son los que no tienen tests unitarios propios, que quedan cubiertos indirectamente por los tests de integración del gateway.

La cobertura del gateway alcanza un **88.83 % de sentencias, 72.72 % de ramas, 92 % de funciones y 89.9 % de líneas**. Las rutas tienen cobertura del 100% y los middlewares también. El único componente con baja cobertura es el controlador geográfico `geoController.js`, que actúa como proxy hacia PyGeoAPI y cuya lógica es difícil de testear sin un servidor geográfico real en marcha.

El objetivo de cobertura establecido era superar el 50 % del código de backend. Tanto el app-server como el gateway lo superan con holgura, alcanzando el app-server un 79.32 % y el gateway un 88.83 % de cobertura de sentencias. En las siguientes figuras se muestra la salida del comando `npm run test:coverage` ejecutado en cada uno, donde se puede comprobar el número de tests pasados y los porcentajes de cobertura obtenidos.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	79.32	70.98	76.82	80.25	
application/use-cases	75	69.53	64.28	75.18	
CancelarReservaPropia.js	100	100	100	100	
CrearUsuario.js	100	100	100	100	
EliminarReserva.js	100	91.3	100	100	66,96
GetEspaciosMetadatos.js	0	0	0	0	18-54
GetNotificaciones.js	0	0	0	0	17-74
Login.js	100	100	100	100	
MarcarNotificacionesLeidas.js	0	0	0	0	15-32
ModificarEdificio.js	96.77	79.62	71.42	96.61	121,167
ModificarEspacio.js	69.87	63.59	62.5	70.71	36-37,134,201,262-325,354-374,395
ModificarUsuario.js	100	97.14	100	100	80
ObtenerReservasUsuario.js	0	0	0	0	15-45
ObtenerReservasVivas.js	0	0	0	0	21-115
ObtenerUsuario.js	100	100	100	100	
ReservarEspacio.js	81.96	68.42	100	81.19	104,166-169,215,224-245,257-259
domain/entities	68.36	61.22	71.05	72.43	
Departamento.js	0	0	0	0	26-47
Edificio.js	0	0	0	0	36-86
Espacio.js	97.72	81.39	90.9	97.61	95
Notificacion.js	64.28	33.33	33.33	72	63,89-111
Reserva.js	100	100	100	100	
Usuario.js	66.66	64.28	40	75	41,47,70-93
domain/factories	100	30	100	100	
NotificacionFactory.js	100	33.33	100	100	32-37
ReservaFactory.js	100	50	100	100	37-38
UsuarioFactory.js	100	0	100	100	34
domain/policies	93.93	92.59	100	96	
ReservaPolicy.js	93.93	92.59	100	96	93
domain/services	97.32	80.95	82.35	97.93	
InvalidacionReservasService.js	97.02	80.48	78.57	97.77	108,191
SolapamientoService.js	100	100	100	100	
domain/value-objects	95.57	80.72	97.22	97	
CategoriaReserva.js	88.63	72.91	92.3	91.89	119,148-149
PeriodoTiempo.js	100	100	100	100	
Rol.js	100	75	100	100	122-124
Test Suites: 17 passed, 17 total					
Tests: 453 passed, 453 total					
Snapshots: 0 total					
Time: 3.82 s, estimated 5 s					

Figura 20: Cobertura de tests del app-server

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	88.83	72.72	92	89.9	
controllers	81.73	64.7	87.5	81.73	
authController.js	100	100	100	100	
edificioController.js	100	100	100	100	
espaciosController.js	100	100	100	100	
geoController.js	12.5	14.28	0	12.5	10-44
notificacionesController.js	91.66	100	100	91.66	19
reservasController.js	87.5	100	100	87.5	22, 33, 47, 62
usuariosController.js	90.47	100	100	90.47	29, 39
dtos	87.5	80	100	96.15	
createReservaDto.js	85.71	75	100	100	4, 10-13, 24-30
loginDto.js	90.9	91.66	100	90.9	17
middlewares	100	70	100	100	
authMiddleware.js	100	100	100	100	
errorHandler.js	100	50	100	100	2-4
validateDto.js	100	50	100	100	8
routes	100	100	100	100	
authRoutes.js	100	100	100	100	
edificioRoutes.js	100	100	100	100	
espaciosRoutes.js	100	100	100	100	
geoRoutes.js	100	100	100	100	
notificacionesRoutes.js	100	100	100	100	
reservasRoutes.js	100	100	100	100	
usuariosRoutes.js	100	100	100	100	
services	100	75	100	100	
jwtService.js	100	75	100	100	10
Test Suites: 7 passed, 7 total Tests: 58 passed, 58 total Snapshots: 0 total Time: 3.168 s Ran all test suites.					

Figura 21: Cobertura de tests del gateway

3.7. Calidad y construcción automática

3.7.1. GitHub Actions CI

El sistema de integración continua se implementa mediante GitHub Actions. El pipeline se ejecuta automáticamente en cada push o pull request a la rama main, y está dividido en dos jobs independientes que se ejecutan en paralelo: uno para el app-server y otro para el gateway.

Cada job instala Node.js 20, instala las dependencias con `npm ci` y ejecuta `npm run test:coverage`. Si algún test falla el pipeline se detiene y el pull request no puede mergearse. El gateway requiere además las variables de entorno `JWT_SECRET` y `JWT_EXPIRES_IN`, que se inyectan desde los secrets del repositorio de GitHub.

3.7.2. Estrategia de ramas y definición de hecho

Los repositorios de backend y frontend siguen una estrategia de ramas por desarrollador. Cada miembro del equipo trabaja en su propia rama y cuando termina una funcionalidad abre un pull request hacia la rama main. El repositorio de documentación tiene únicamente la rama main ya que no requiere revisión de cambios.

La rama main del repositorio de backend está protegida. Para que un pull request pueda mergearse es necesario que al menos otro miembro del equipo lo revise y apruebe,

además de que el pipeline de CI pase correctamente. Esto es así porque el backend contiene toda la lógica de negocio del sistema y un error ahí puede romper el funcionamiento completo de la aplicación. En el frontend la protección es menos estricta ya que cada miembro puede aceptar sus propios pull requests, aunque sí se revisan los cambios antes de mergear, ya que un error en el frontend tiene un impacto más localizado y fácil de detectar.

Definición de hecho

Una tarea se considera completada cuando el código está implementado, revisado mediante pull request por al menos un miembro del equipo, los tests correspondientes pasan correctamente y la funcionalidad está documentada.

3.7.3. Gestión de issues

Con el objetivo de mantener una trazabilidad clara entre los requisitos del sistema y el trabajo realizado, se han creado issues en GitHub para cada una de las funcionalidades implementadas. Cada issue recoge el título de la tarea, los requisitos que cubre y las subtarefas concretas a realizar. Dado que la mayoría de funcionalidades implican trabajo tanto en el backend como en el frontend, se ha optado por centralizar todos los issues en el repositorio de documentación, evitando así duplicarlos en los repositorios de backend y frontend por separado. Cada issue indica explícitamente si su implementación afecta únicamente al backend, únicamente al frontend, o a ambas partes del sistema.

4. Conclusiones

4.1. Resumen

El proyecto ByronSpace ha permitido al equipo desarrollar un sistema de reserva y gestión de espacios completo y funcional para el edificio Ada Byron de la EINA. A lo largo del desarrollo se ha trabajado con una arquitectura real de varios servicios comunicados de forma asíncrona, aplicando los principios del Diseño Dirigido por el Dominio en la capa de negocio y siguiendo buenas prácticas de calidad como la integración continua y los tests automáticos.

El resultado es una aplicación web operativa que cubre toda la funcionalidad requerida y cuatro puntos de funcionalidad opcional, con una interfaz gráfica moderna que integra un mapa interactivo y un backend robusto con más de 500 tests automatizados.

El desarrollo del proyecto ha supuesto un reto técnico considerable, especialmente en lo que respecta a la arquitectura hexagonal, la comunicación asíncrona mediante RabbitMQ y la integración del sistema de información geográfica con PostGIS y PyGeoAPI. Estos desafíos han permitido al equipo consolidar conocimientos técnicos y adquirir experiencia práctica en el desarrollo de sistemas software de cierta envergadura.

4.2. Cumplimiento de objetivos

El equipo considera que el proyecto cumple los requisitos para obtener la calificación de sobresaliente. A continuación se justifica el cumplimiento de cada objetivo de evaluación.

Objetivos de aprobado

El proyecto se aloja en la organización de GitHub en la URL <https://github.com/UNIZAR-30249-2026-RESERVASADA>, donde se puede comprobar el historial de commits y el flujo de trabajo con ramas y pull requests. La compilación y gestión de dependencias se gestionan mediante scripts de npm definidos en los archivos `package.json` de cada servicio, concretamente en las carpetas `app-server/`, `gateway/` del repositorio backend y `frontend/` del repositorio frontend.

El control de esfuerzos con las horas dedicadas por persona se recoge en la sección 4.3 de este documento.

La aplicación cumple toda la funcionalidad requerida, detallada en el catálogo de requisitos del capítulo 2 y en el apartado 3.1 de este documento. Además se han implementado cuatro puntos de funcionalidad opcional: O1, O2, O3 y O7, descritos en la sección 2.2.5.

La documentación arquitectural incluye una vista de módulos (sección 3.2.1), una vista de componentes y conectores (sección 3.2.2) y una vista de despliegue (sección 3.2.3), reflejando fielmente el sistema implementado.

El estilo arquitectural de módulos del `app-server` es hexagonal, con tres capas bien diferenciadas: `domain`, `application` e `infrastructure`, ubicadas en la carpeta `app-server/src/` del repositorio backend y documentadas en la sección 3.2.1.

El estilo arquitectural de componentes y conectores es N-Tier con 4 niveles: frontend React (N1), gateway Node+Express (N2), servidor de aplicaciones y PyGeoAPI con RabbitMQ (N3) y base de datos PostgreSQL+PostGIS (N4), documentado en la sección 3.2.2.

El Diseño Dirigido por el Dominio se aplica correctamente en la capa de dominio del app-server, con entidades, objetos valor, agregados, factorías y repositorios implementados en la carpeta app-server/src/domain/ y documentados en las secciones 3.3.1, 3.3.2 y 3.3.3.

Objetivos de notable

La cobertura de tests automáticos del backend supera el 50 % del código. El app-server alcanza un 79.32 % de cobertura de sentencias y el gateway un 88.83 %, medidos con el comando `npm run test:coverage` que ejecuta Jest con la opción `-coverage`. Los resultados se muestran en las figuras de la sección 3.6.4. Los tests se encuentran en las carpetas app-server/tests/ y gateway/tests/ del repositorio backend.

Se ha puesto en marcha un servicio de integración continua mediante GitHub Actions que ejecuta automáticamente todos los tests del backend en cada push o pull request a la rama main, como se describe en la sección 3.7.1. El fichero de configuración se encuentra en .github/workflows/ del repositorio backend.

El modelo de dominio utiliza correctamente los conceptos de servicios, interfaces reveladoras, aserciones y funciones sin efectos secundarios, implementados en app-server/src/domain/ y documentados en la sección 3.3.4.

Se han implementado cuatro puntos de funcionalidad opcional, superando los dos requeridos para notable: O1, O2, O3 y O7.

Objetivos de sobresaliente

Toda la comunicación entre el nivel 2 (gateway) y el nivel 3 (servidor de aplicaciones) se produce mediante mensajes asíncronos con RabbitMQ como broker de mensajes, implementando el patrón RPC sobre AMQP. La implementación se encuentra en gateway/src/messaging/ y gateway/src/services/appServerMessagingClient.js por parte del gateway, y en app-server/src/infrastructure/messaging/ por parte del servidor de aplicaciones. Se describe en detalle en la sección 3.2.2.

Se han implementado cuatro puntos de funcionalidad opcional, superando el punto adicional requerido para sobresaliente: O1 (porcentaje de ocupación configurable por espacio), O2 (resultados de búsqueda sobre mapa interactivo), O3 (despachos asignados a departamento reservables por investigadores y docentes del mismo departamento) y O7 (nuevo rol de investigador visitante con despacho asignable).

4.3. Horas dedicadas por persona

A continuación se detallan las horas dedicadas por cada miembro del equipo a lo largo del proyecto, desglosadas por quincena y tarea. La información completa también puede consultarse en el documento de recogida de información.

Período	Horas	Tareas
5 feb – 20 feb	8h	Elaboración del informe preliminar.
20 feb – 6 mar	10h	Corrección del informe preliminar y toma de decisiones de diseño.
8 mar – 23 mar	30h	Desarrollo de estructura de frontend y backend. Carga de datos GeoJSON en PostGIS y subida al servidor geográfico PyGeoAPI. Visualización del mapa y espacios en el cliente web. Funcionalidad de login y creación de reservas. Documentación de APIs en Swagger. Desarrollo del informe de la primera iteración.
24 mar – 6 abr	15h	Desarrollo de la comunicación entre gateway y servidor de aplicaciones mediante broker. Mejora de APIs y reglas de negocio. Desarrollo e implementación de los tests de backend. Implementación de GitHub Actions.
7 abr – 20 abr	20h	Reorganización del modelo de dominio (factorías, servicios, interfaces reveladoras, aserciones, funciones sin efectos secundarios). Funcionalidad de visualizar reservas propias. Modificación de crear reserva con más de un espacio. Desarrollo de la presentación intermedia. Diagrama de clases de dominio y diagrama de paquetes del servidor de aplicaciones.
21 abr – 4 may	40h	Desarrollo en frontend y backend de todas las gestiones del gerente: reservas vivas, gestión de espacios (categorías, reservable, asignaciones, horarios, porcentaje de ocupación), gestión de usuarios (roles, asignación de despacho). Desarrollo de las invalidaciones de reservas y notificaciones a usuarios.
5 may – 20 may	30h	Mejora estética de pantallas. Solución de errores. Finalización de funcionalidades. Mejora del código para cumplir con el DDD. Documentación exhaustiva del servidor de aplicaciones. Pruebas mediante peticiones desde terminal.
20 may – 30 may	25h	Puesta a punto del servidor de aplicaciones. Creación de la vista de módulos y despliegue. Modificación del diagrama de clases y del autómata del ciclo de vida de una reserva. Creación y redacción de la memoria en L ^A T _E X. Finalización de los tests unitarios y de integración.

Período	Horas	Tareas
Total: 178h		

Tabla 10: Horas dedicadas por Mario Caudevilla Ruiz

Período	Horas	Tareas
5 feb – 20 feb	10h	Elaboración del informe preliminar.
20 feb – 6 mar	8h	Corrección del informe preliminar y toma de decisiones de diseño. Análisis de requisitos y definición de la tabla de roles y espacios.
8 mar – 23 mar	20h	Desarrollo de la estructura inicial del frontend con React y Vite. Implementación de la pantalla de login y gestión de sesión con JWT. Primer prototipo del formulario de reserva. Configuración inicial de Docker Compose.
24 mar – 6 abr	10h	Implementación parcial de los controladores y rutas del gateway. Middleware de autenticación JWT y gestión de errores.
7 abr – 20 abr	10h	Visualización de espacios del edificio Ada Byron con colores por categoría. Implementación del buscador con filtros por planta, categoría y ocupantes.
21 abr – 4 may	10h	Panel de reservas propias del usuario. Panel de notificaciones con indicador de no leídas. Tarjeta informativa del usuario con rol y permisos.
5 may – 20 may	18h	Mejora estética de pantallas y adaptación responsive. Solución de errores en la integración frontend-gateway. Pruebas funcionales de todos los flujos de la aplicación. Ajuste de Docker Compose y variables de entorno.
20 may – 30 may	8h	Redacción y maquetación de la memoria en $\text{\LaTeX}$. Preparación de la presentación final. Revisión general de la aplicación.
Total: 94h		

Tabla 11: Horas dedicadas por Víctor Martínez Páramo